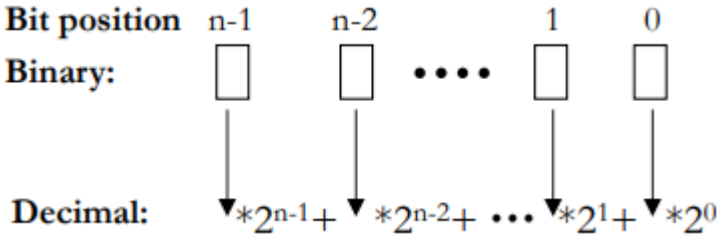
Carl Kaziboni

Computer Systems

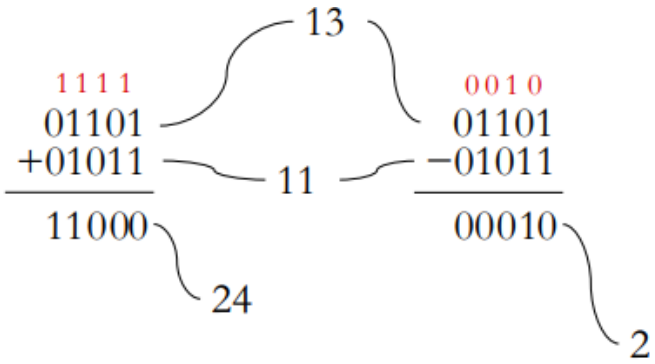
2024

Lecture 1-3

- Sequences of symbols represent information
- Bit – Binary digit
- Advantages- easy computation, very reliable, simple& reusable circuits
- Disadvantages – Little information per bit -> must use many bits



- Basic operations:



  - Don't forget to carry the ones
- Hardware cannot handle infinitely long bit sequences
- Fixed-size data types
  - Byte – 8 bits
  - Word: the typical unit of data on which a processor operates (32 or64 bits most common)
  - Overflow happens when a result does not fit
- Negative numbers
  - Have a sign of magnitude, which uses Most significant bit as the sign, 0 -> positive, 1 -> negative
    - 0100 = 2, 1010=-2
  - Complicates addition and subtraction ☹️
  - Has positive and negative zero ☹️
  - Better way: 2's complement representation
- Two's complement Approach
  - Represent the negation of X as 2^N-X

- Then: X+(-X) = X + (2^N-X) = 2^N
  - Largest number represented with N bits: 2^N-1

Given:

- 3-bit fixed width (N=3)
- X = 2 (decimal) → 0 1 0 (binary)

2^N = 8 (dec) → 1 0 0 0 (bin)

-X = 2^N – X = 8 – 2 = 6 (dec) → 1 1 0 (bin)

Check:

X + (-X) = 0 1 0 + 1 1 0 = 1 0 0 0

3-bit fixed width

- to get -2 (110)
Bits:	1	0	1	0
Decimal bit value:	−8	4	2	1
Binary calculation:	−(1×2^3)	(0×2^2)	(1×2^1)	(0×2^0)
Decimal calculation:	−(1×8)	0	1×2	0

  for -6 (1010)

- Efficiently computing 2's complement
  - Flip the bits and add 1, ignore any overflow

Example:

X = 0 1 0 (bin) → 2 (dec)

Flip the bits: 1 0 1

Add 1: 1 1 0 → -X (bin)

- 2's complement details
  - The MSB is the sign
  - A-B = A + (2's complement of B)
  - Arithmetic operations do not depend on the operands signs
  - Range is asymmetric: -2^{n-1} to 2^{n-1}-1 ☹
    - Given a 4 bit word the largest is 0111(7), smallest is 1000(-8)
  - Two kinds of overflow – Positive overflow produces a negative number, negative underflow produces a positive number
- Coverting between data types:

- Converting a 2's complement number from a smaller to a larger representation is done by sign extension
- Example: from byte to short (16 bits):

$2 = 0\,0\,0\,0\,0\,0\,1\,0 \Rightarrow ?\,?\,?\,?\,?\,?\,?\,?\,0\,0\,0\,0\,0\,0\,1\,0$

$-2 = 1\,1\,1\,1\,1\,1\,1\,0 \Rightarrow ?\,?\,?\,?\,?\,?\,?\,?\,1\,1\,1\,1\,1\,1\,1\,0$

$2 = \underbrace{00000010}_{\text{(byte)}} \Rightarrow \underbrace{0000000000000010}_{\text{(short)}}$ $-2 = \underbrace{11111110}_{\text{(byte)}} \Rightarrow \underbrace{1111111111111110}_{\text{(short)}}$

- Shifting
  - Shifting: move the bits of a data type left or right
    - Data bits "falling off the edge" are lost
  - For left shifts, 0s fill in the empty bit places
  - For right shifts, two options:
    - Fill with 0 (logical shift): for non-numerical data
    - Fill with MSB (arithmetic shift): for 2's complement numbers
  - Shift left by n is equivalent to multiplying by 2^n
  - Shift right by n is equivalent to dividing by 2^n and rounding towards $-\infty$
  - Example

$6 = 0\,0\,0\,0\,0\,1\,1\,0 << 2 \rightarrow 0\,0\,0\,1\,1\,0\,0\,0 = 24$

$6 = 0\,0\,0\,0\,0\,1\,1\,0 >> 2 \rightarrow 0\,0\,0\,0\,0\,0\,0\,1 = 1$

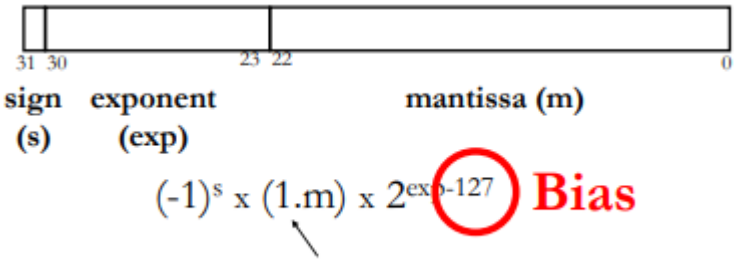
$-6 = 1\,1\,1\,1\,1\,0\,1\,0 >> 2 \rightarrow 1\,1\,1\,1\,1\,1\,1\,0 = -2$

- Hexadecimal notation
- Binary numbers (and other binary-encoded information) are too long for humans to use, hexadecimal is more compact
- Hexadecimal (base 16) is most common, Hex digits: 0-9,A-F
  - A=10, B=11,…,F=15
- Conversion to/from binary is
  - Every 4 bits correspond to 1 hex digit (grouped from right to left)

$\underbrace{1\,1\,1\,1}_{\text{F(15)}}\underbrace{1\,0\,0\,0}_{8} = 0xF8$

- Representing floating points
  - Example 0.75 = 0.11 ($2^{-1} + 2^{-2} = 0.5 + 0.25 = 0.75$)

    - In normalized form would be 0.11-> 1.1×2^{-1}
- Normalising reasons
    - Simplifies machine representation
    - Simplifies comparisons
    - Is more compact for very large numbers
- IEEE 754 Floating Point Standard
    - 32 bit representation



- $(0.75)_{10} \rightarrow (0.11)_2 \rightarrow (1.1 \times 2^{-1})_2$
  $\rightarrow$ s = 0, m = 1, exp = 126
  $\rightarrow$ 0 01111110 10000000000000000000000
- (pad the end with 0s)

Exponent	Mantissa	Meaning
0	0	0
1-254	Anything	Floating point number
255 (0xFF)	0	Infinity (signed)
255 (0xFF)	Non-zero	Not-a-number (NaN)

32-bit representation

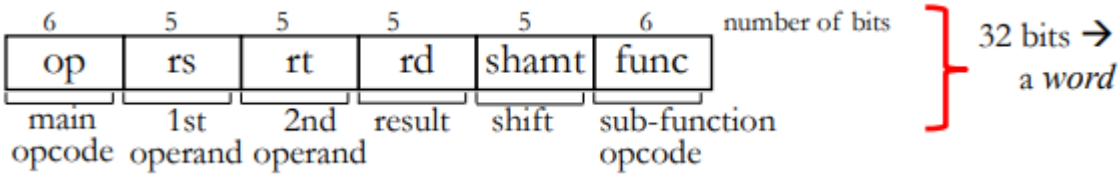
- Characters are also encoded in binary, most common representation is ASCII
- Java uses Unicode which can encode characters from all languages, 16 bits per character
- Strings are represented in different ways, C with null character, java keeps length of string itself

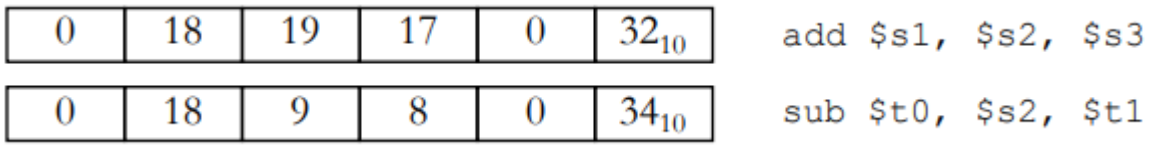
Lecture 4-6

- Instruction set architecture (ISA): the interface between the software and the hardware

    - Collection of all machine instructions recognized by a particular processor
    - Privilege levels, memory management, etc.
- The ISA abstracts away the hardware details from the programmer
- Instructions are just strings of binary numbers
- Assembly language: symbolic representation of machine instructions
    - Assembly language makes it easy for humans to read and write machine code
    - There is a 1-to-1 mapping between a machine instruction and a corresponding assembly instruction
- MIPS assembly simple example:
    - a[0] = b[0] + 10
    - lw r4, 0(r2) #get the value of b[0] from memory and store it in register r4
      addi r5, r4, 10 #compute b[0] + 10 and store into r5
      sw r5, 0(r1) #store r5 into a[0]
    - Separate instructions to access data (in memory) % operate on it
    - MIPS does not allow to operate on memory directly
- MIPS arithmetic & logical operations:
    - Data processing looks like
        - Operation, destination, 1st operand, 2nd operand
    - add a,b,c a = b+c
    - sub a,b,c a = b-c
    - Bit-wise logical instructions: and, or, xor
    - Shift instructions:
        - sll a,b,shamt a = b << shamt
        - srl a,b,shamt a = b >> shamt (logical)
        - sra a,b,shamt a = b >> shamt (arithmetic
- Registers
    - ISA places restrictions on instruction operand
    - MIPS processors operate on registers only
        - Registers are storage locations inside the processor that hold program variables and control state
    - Registers are generally sized to contain machine's word
        - 32 or 64 bits common today
    - Processors have relatively few programmer exposed registers

        - MIPS has 32
        - X86 has 8-16
- MIPS instruction example
    - add $s1, $s2, $s3 #$s1 = $s2 + $s3
    - Binary (R-format used for arithmetic instructions





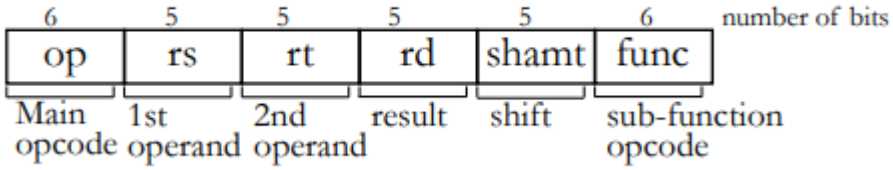
- Most registers are available for programmer use
- Program variables are held in regs $s0-$s7
- Temporary variables (inside a method call): $t0-$t9
- Registers with special roles
    - Register 0 ($zero) is hardwired to 0
    - Program Counter (PC) holds address of next instruction to be executed

Name	Register Number	Usage	Preserved on call
$zero	0	the constant value 0	n.a.
$at	1	reserved for the assembler	n.a.
$v0-$v1	2-3	value for results and expressions	no
$a0-$a3	4-7	arguments (procedures/functions)	yes
$t0-$t7	8-15	temporaries	no
$s0-$s7	16-23	saved	yes
$t8-$t9	24-25	more temporaries	no
$k0-$k1	26-27	reserved for the operating system	n.a.
$gp	28	global pointer	yes
$sp	29	stack pointer	yes
$fp	30	frame pointer	yes
$ra	31	return address	yes

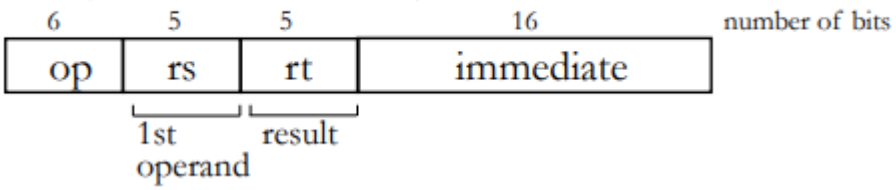
- Immediate operands:
    - MIPS has dedicated instructions with one constant (immediate) operand
    - E.g addi $r1, $r1, 1 #r1 = r1 + 1
    - General from: opi $r1, $r2, n
- Loading a constant operand:

    - Addi $0, $zero, n #n%s0=n
    - For large/smaller constant, assembler pseudo-instruction li reg, constant
    - Translated into 1 instruction for immediates < 16 bits and into more instructions for more complicated cases (e.g. 2 instructions for a 32 bit immediate)
    - lui $s1, n1 #$s1$_{31-16}$=n1; $s1$_{15-0}$=0
      ori $s1, $s1, n2 #$s1$_{31-16}$ = n1; $s1$_{15-0}$=n2
- g = h +A[0] where h1 is in register $s1, A[0] is the first element of array A and is pointed to by $s2
    - MIPS:
        - lw $t1, 0($s2) #$t1 = memory [0 + $s2], here 0 is the offset
        - add $t2, $s1, $t1 # t2 = h + $t1
- Data-transfer instructions
    - Load word:
        - lw r1, n(r2) #r1 = memory[n+r2]
        - sw r1, n(r2) #memory[n+r2] = r1
        - lb r1, n(r2) #r1$_{7-0}$ = memory[n+r2], r1$_{31-8}$ = sign extension
        - sb r1, n(r2) #memory[n+r2] = r1$_{7-0}$ no sign extension
- Memory is byte addressable, but it is organised so that a whole word can be accessed directly
- A word can be stored anywhere (unaligned) or at an address that is a multiple of the word size(aligned), MIPS requires alignment, x86 doesn't
- Instruction representation composed of bit fields
- Similar instructions have same format:

R-format (add/sub, and/or, sll/slr, ...)



I-format (addi, lw, sw, ...)

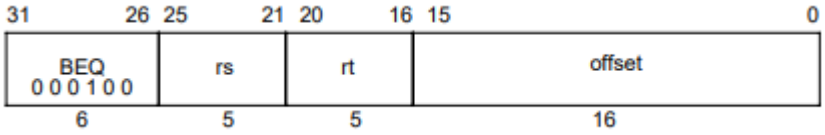


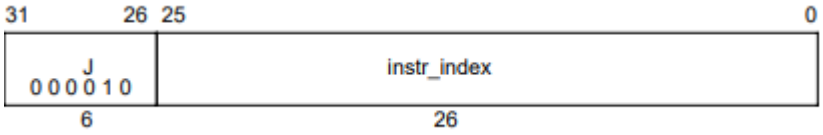
    -
- Example – swap array elements

- void swap(int v[], int ind)
  {
  int temp;
  temp = v[idx];
  v[idx] = v[idx+1];
  v[idx + 1] = temp;
  }
- swap:
  #inputs: $a0 – array base, $a1 - index
  sll $t0, $a1, 2 #reg $t0 = idx * 4 (multiply by 4 is shifting by two to the right in binary)
  add $t0, $a0, $t0 #reg $t0 = v + (idx * 4), #$t0 holds the address of v[idx]

  #Load the two values to be swapped
  lw $t1, 0($t0) #reg $t1 = v[idx]
  lw $t2, 4($t0) # reg $t2 = v[idx + 1]

  #Store the swapped values back to memory
  sw $t2, 0($t0) # v[idx] = $t2
  sw $t1, 4($t0) # v[idx + 1] = $t1
- Control transfer: If structures
  - C
    - if (i != j)
      stmnt1
      else
      stmnt2
      stmnt3
  - MIPS: "branch if equal": beq $s1, $s2, label
  - Compare value in $s1 with value in $s2
  - If equal, branch to instruction marked label
    - beq $s1, $s2, label2
      stmnt1
      j label3 #skip stmnt2, j is unconditional jump
      label2: stmnt2
      label3: stmnt3

- Conditional branches, I format:
  - 

  - Assembly, label is usually a string
  - Machine code, label is a PC-relative offset
  - Obtained from immediate operand as
    - Branch target = PC + 4 * offset
  - Similarly: bne r1, r2, label #if r1! = r2 go to label
  - Unconditional jump, j-format: j label
    - 

  - C ; while (count != 0) stmnt
  - MIPS:
    - loop:

      beq $s1, $zeero, end #$s1 holds count stmnt

- Comparisons
  - "Set if less than" (R-format): slt r1, r2, r3
    - Set r1 to 1 if r2 < r3, otherwise set r1 to 0
  - C: while (i > j) stmnt
  - MIPS:
    - loop:

      slt $t0, $s2, $1 #$t0 = (j < i)

      beq $t0, $zero, end #branch if i <= j

      stmnt

      j loop # jump back to loop

      end: …

C operator	Comparison	Reverse	Branch
==	seq	0	bne
!=	seq	0	beq
<	slt, sltu, ..	0	bne
>=	slt, sltu, ..	0	beq
>	slt, sltu, ..	1	bne
<=	slt, sltu, ..	1	beq

  -

- MIPS Instruction Format Summary
  - R-type (register to register)
    - Three register operands
    - Most arithmetic, logical and shift instructions
  - I-type (register with immediate)
    - Instructions which use two registers and a constant
    - Arithmetic/logical with immediate operand
    - Load and store
    - Branch instructions with relative branch distance
  - J-type (jump)
    - Jump instructions with a 26 bit address
- Method calls
  - Jumping into the method: jal label
    - "jump and link"
    - Set $ra to PC + 4
    - Set PC to label
    - J-format instruction
  - Returning: jr ra
    - "jump register": set PC to value in register $ra
    - Note that any register can be used as a jump target in jr
- MIPS register convention on method calls:
  - Method parameters: $a0 - $a3
  - Return values: $v0, $v1
  - Regs preserved across call boundaries: $s0 - $s7
  - Regs not preserved across call boundaries: $t0-$t9
- Nested calls -> must save a return address to prevent overwriting.
  Solution: use a stack in memory
- To push a word:
  - addi $sp, $sp, -4 #move sp down
  - sw $ra, 0($sp) #save ra on top of stack
- To pop a word :
  - lw $ra, 0($sp)
  - addi $sp, $sp, 4
  - sp always points to the last valid word on the stack
- Other uses of the stack

- Stack used to save caller's registers, so that they can be used by the callee
  - In MIPS, these are the $t registers
  - General issue: "caller save" vs "callee save"
- Stack can also be used to pass & return parameters
  - Gets around the limited number of parameter and return the value registers
- Finally stack is used for local variables within the function

- Stack frame – a region of a stack used within a function call
- Frame pointer $fp: points to the base of the stack frame
  - Enables "fixed" addressing within a stack frame despite the stack pointer ($sp) moving around
  - Saved on a function call
- Swapping elements finishing:

```
int main(void) {
  int size = 6;
  int v[] = {1, 10, 2, 20, 3, 30};  // array w/ 6 elements

  for (int i=0; i<size; i+=2)
    swap(v, i);  // pass array (by reference) and index i
}
```

  - MIPS :
    - main :

```
la $s0, array #$s0 : base address of v[]
addi $s1, $zero, 0 #$s1 : index into the array (i)
addi $s2, $zero, 6 #$s2 : array size

loop :
beq $s1, $s2, out
move $a0, $s0 #$a0 = $s0 (array base pointer)
move $a1, $s1 #$a1 =$s1 (index)
jal swap

addi $s1, s1, 2 #incrememnt the index
j loop
```

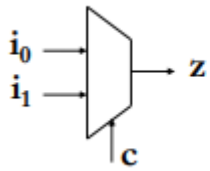
out :

```
swap:
        # Compute the address into the array
        sll $t0, $a1, 2     # reg $t1 = idx * 4
        add $t0, $a0, $t0   # reg $t1 = v + (idx*4)
                            #  $t1 holds the addr of v[idx]

        # Load the two values to be swapped
        lw  $t1, 0($t0)     # reg $t0 = v[idx]
        lw  $t2, 4($t0)     # reg $t2 = v[idx+1]

        # Store the swapped values back to memory
        sw  $t2, 0($t0)     # v[idx] = $t2
        sw  $t1, 4($t0)     # v[idx+1] = $t0

        jr  $ra             # return to main
```

- ISA range in complexity
- MIPS is a relatively simple ISA
- Example
    - a[0] = b[0] + 10
    - In Complex assembly : ADDW3 (R5), R2), 10 #VAX instruction
- Complex Instruction Set (CISC)
    - Appeared in early computers
    - Computers programmed in assembly -> high-level language features as instructions (e.g) x86 instructions to manipulate strings
    - Very few registers -> operands can be in memory
    - Very little memory -> variable length instructions to minimize code size
- Reduced Instruction Set (RISC)
    - Appeared in the 80s. Used in ARM, MIPS, and SPARC ISAs
    - Compilers -> Simple instructions
    - More registers -> load-store architecture
    - More memory & faster clock frequency -> fixed length, fixed format instructions for easy, fast decoding logic

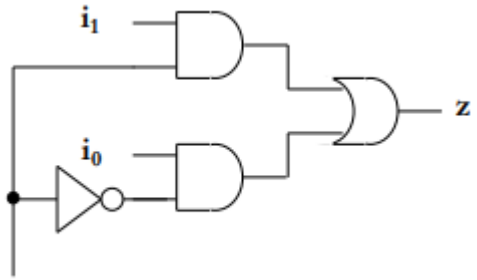
Logic Design

- Binary digital logic circuits:
    - Two voltage levels (ground and supply voltage) for 0 and 1
        - Built from transistors
- Combinational logic: output depends only on current inputs
- Sequential logic: output depends on the current inputs as well
- Combinational Logic Circuits: traditional NOT, AND, OR, NOR, AND and NAND gates
- Functional completeness 0 set of gates that is sufficient to express any Boolean function
- Examples – {AND, OR NOT}, {NAND} and {NOR}
- Multiplexer – a circuit for selecting one of multiple inputs
    - This is a selector, since its output is one of the inputs



$$z = \begin{cases} i_0 & \text{iff } c=0 \\ i_1 & \text{iff } c=1 \end{cases}$$

    -
    - Sum of products in multiplexer: $\bar{c}.i_0 + c.i_1$
    - Can be implemented with inverted, 2 AND gates and 1 OR gate:



        -
    - Sum of products not practical for circuits with large number of inputs (n)
        - The possible products can be proportional to 2^n
    - Programmable logic array (PLA) – a structured-logic element composed of a set of inputs and corresponding input complements and two stages of logic: the first generates product terms of the inputs and input complements and the second generates sum terms of the product terms. Hence, PLAs implement logic functions as a sum of products.

- Minterms/product terms – a set of logic inputs joined by conjunction (AND operations); the product terms form the first logic stage of programmable logic array
- A PLA has two characteristics that help make it an efficient way to implement set of logic functions
  - Only the truth table entries that produce a true value for at least one output have any logic gates associated with them
  - Each different product term will have only one entry in the PLA, even if the product term is used in multiple outputs

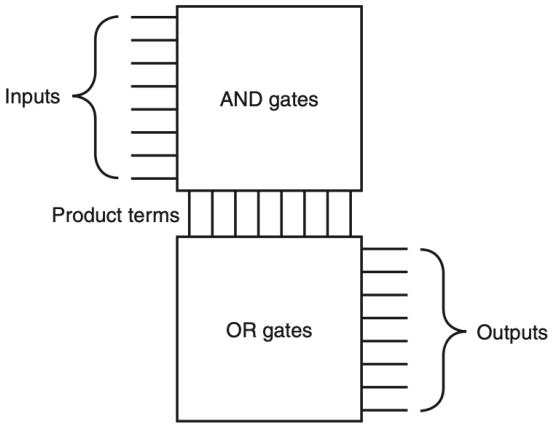
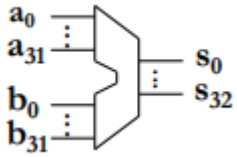
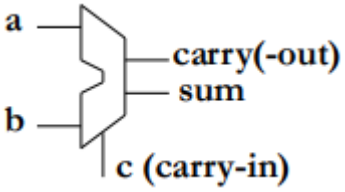


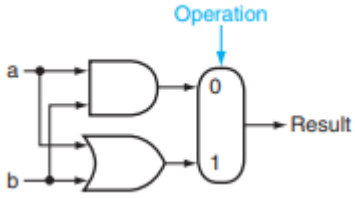
FIGURE B.3.3 The basic form of a PLA consists of an array of AND gates followed by an array of OR gates. Each entry in the AND gate array is a product term consisting of any number of inputs or inverted inputs. Each entry in the OR gate array is a sum term consisting of any number of these product terms.

- 32 bit adder is – 64 inputs- too complex for sum of products



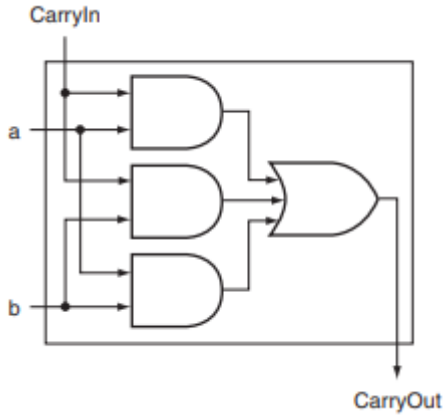
- Full adder:

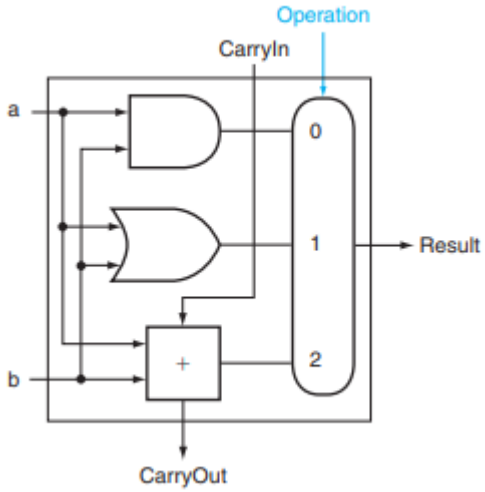




  - 1 Bit logical unit for AND and OR

- $sum = \overline{a}.\overline{b}.c + \overline{a}.b.\overline{c} + a.\overline{b}.\overline{c} + a.b.c$
- $carry = b.c + a.c + a.b$
- Adder hardware for the CarryOut Signal

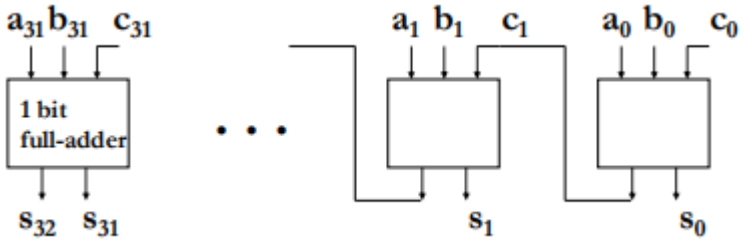


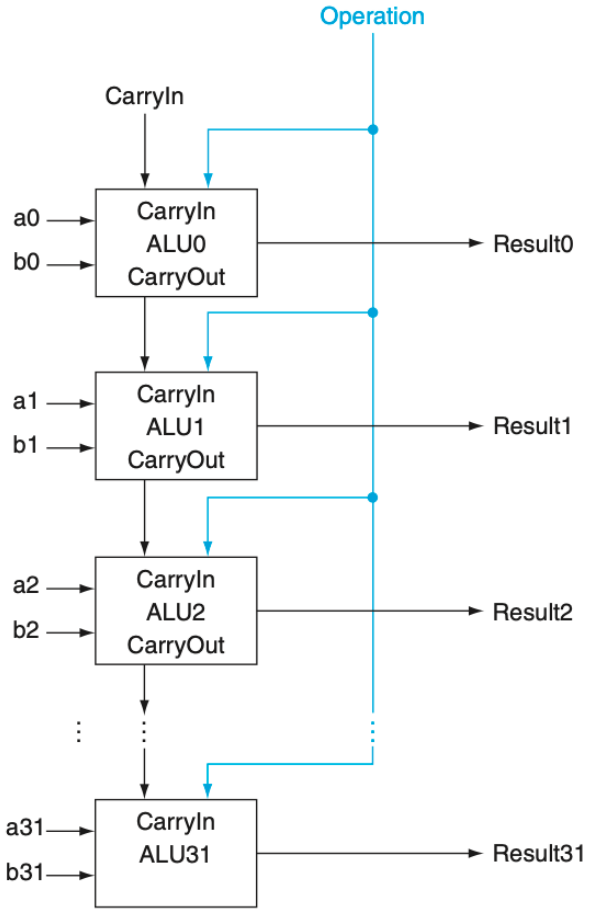


- 1 bit ALU that performs AND OR and addition

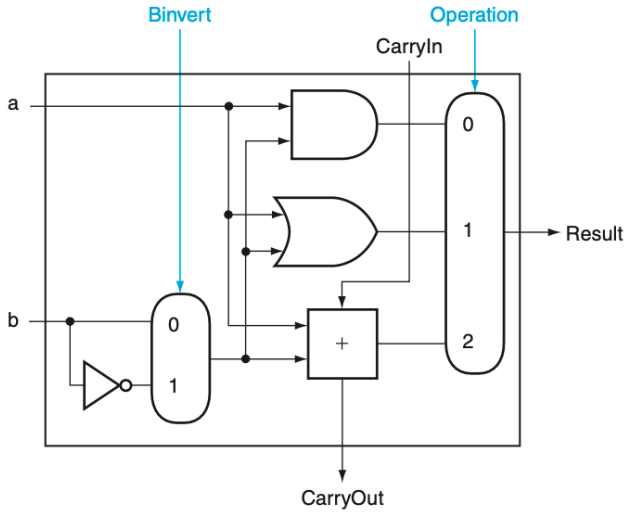
a	b	c	carry	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- Ripple carry adder (go back to ALU in textbook)
  - 32 bit adder: chain of 32 full adders

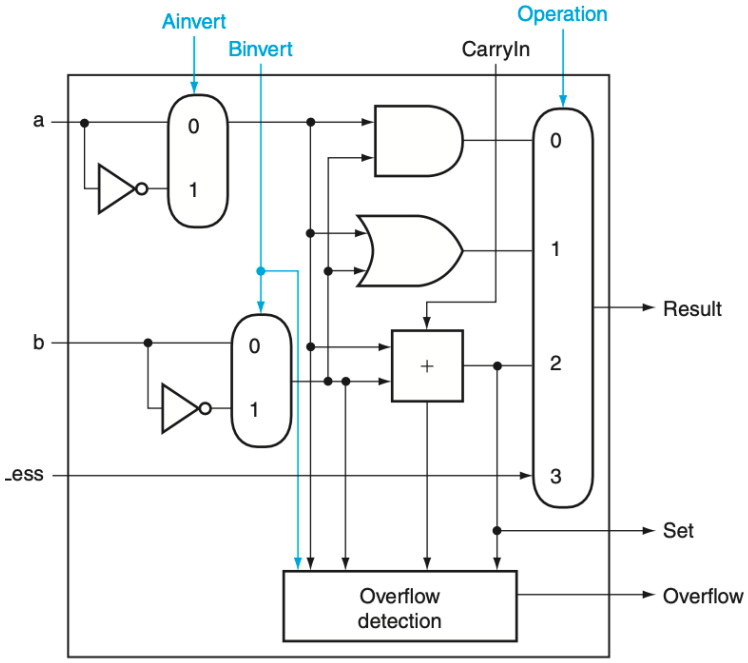




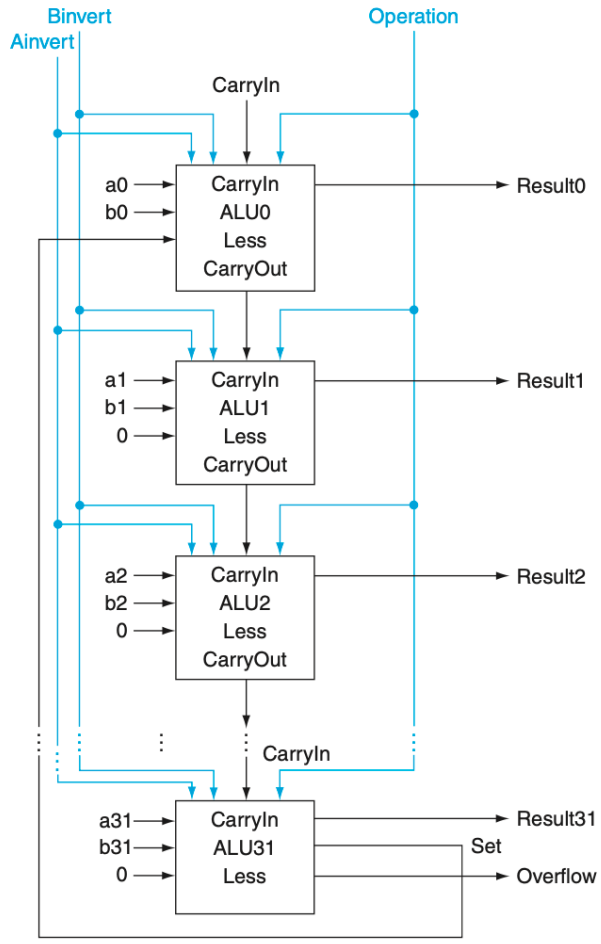
- Multiplexor selects result to get desired operation



- 1 bit ALU that performs AND, OR and addition on and b or a and -b (B-invert to make b negative(subtraction)

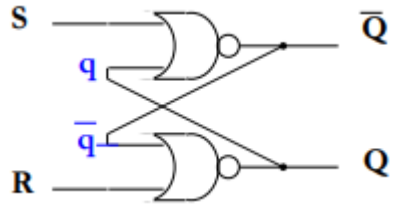


- Performs AND, OR, addition, on a and b or b (invert) and bottom



- 32 bit ALU from 31 copies of 1 bit ALU
- Every time we want the ALU to subtract we set both CarryIn and B-invert to 1
- For adds or logical operations we want both control lines to be 0
- To further tailor ALU to MIPS ISA, we must support conditional branch, testing equality by using, a – b = 0 => a = b
- Easiest way is to OR all the outputs together and then send that signal through an inverter
- $$\text{Zero} = \overline{(\text{Result31} + \text{Result30} + \ldots + \text{Result2} + \text{Result1} + \text{Result0})}$$

- Carry bits c_i are computed in sequence c_1, c_2, …, c_{32} (where $c_{32} = s_{32}$) as c_i depends on c_{i-1}
- Since sum bits s_i also depend on c_i they too are computed in sequence
- Subtraction is the same as adding the negative version of an operand and this is how adders perform subtraction

- Propagation Delay – the time delay between input signal change and output signal change at the other end
- Delay depends on:
    - Technology (transistor parameters, wire capacitance, etc.)
    - Delay through each gate (function of gate type)
    - Number of gates driven by a gates output (fan out)
- E.g: 2 input mux: NOT -> AND -> OR -> 3 gate delays
- Sequential logic has n feedback signals -> up to 2^n states
- Feedback with OR works only once, remembers if there was a 1 in it
- SR latch: the basic sequential circuit
    - 

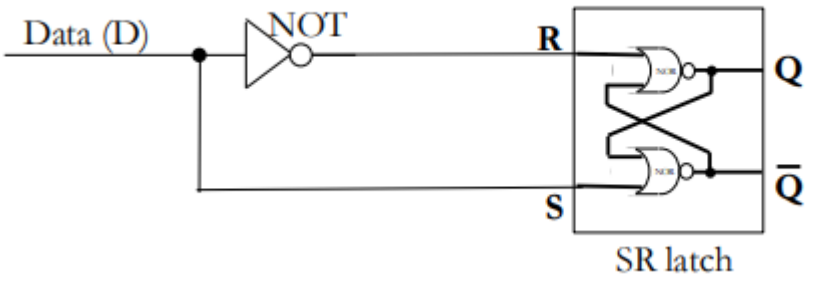
    - Inputs: R, S
    - Feedback: q, $\bar{q}$
    - Output:Q
- Usage: 1 bit memory
    - Two inputs: S and R
    - Outputs are inverse of each other
        - Q is the "current state" of the memory
    - Setting one of the inputs to 1 changes the state
        - Setting S to 1 sets the value Q (0->1)
        - Setting R to 1 resets the value Q (1->0)
        - Keep the value in memory by maintaining S=0 and R=0

S	R	Q_i
0	0	Q_{i-1}
0	1	0
1	0	1
1	1	inv

        - inv – invalid

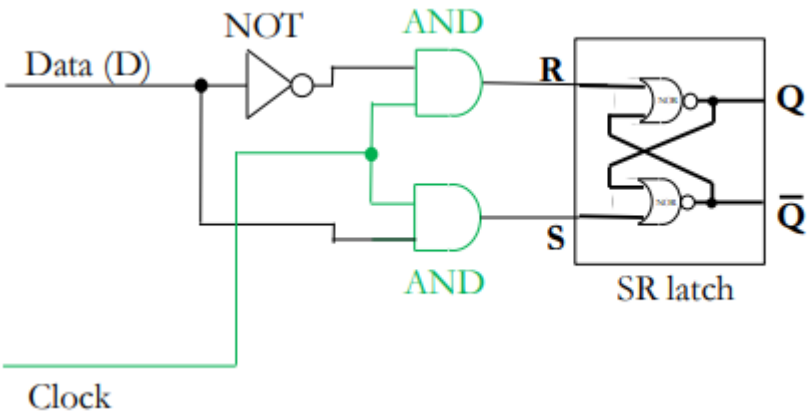
          Set the value in memory to 1 (or 0) by setting S=1 (or R=1) for a short time
- D Latch
    - Want to be able to use SR latch as 1 bit memory element – Not practical with a 2 bit input

- Solution:
  - S = D(input data)
  - R = D'



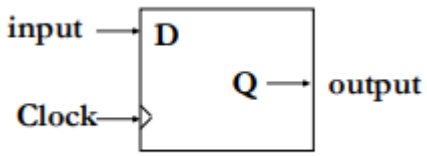
S	R	Q_i
0	0	Q_{i-1}
0	1	0
1	0	1
1	1	inv

- Timing of events
  - Asynchronous sequential logic
    - State (and possibly output) of circuit changes whenever inputs change
  - Problem: different delays on different circuit paths
  - Solution: Synchronous sequential logic
  - State (and possibly output) can only change at times synchronized to an external signal -> the clock
- Clocked D latch

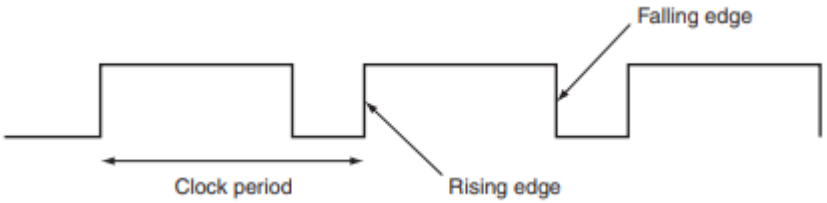


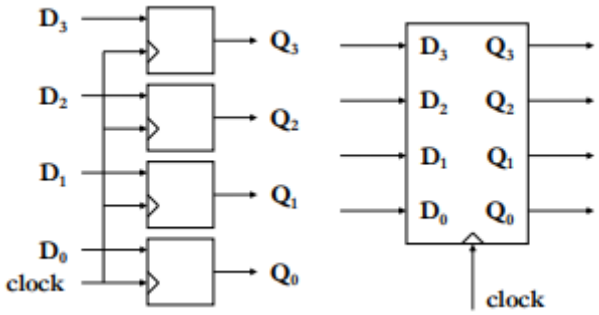
S	R	Q_i
0	0	Q_{i-1}
0	1	0
1	0	1
1	1	inv

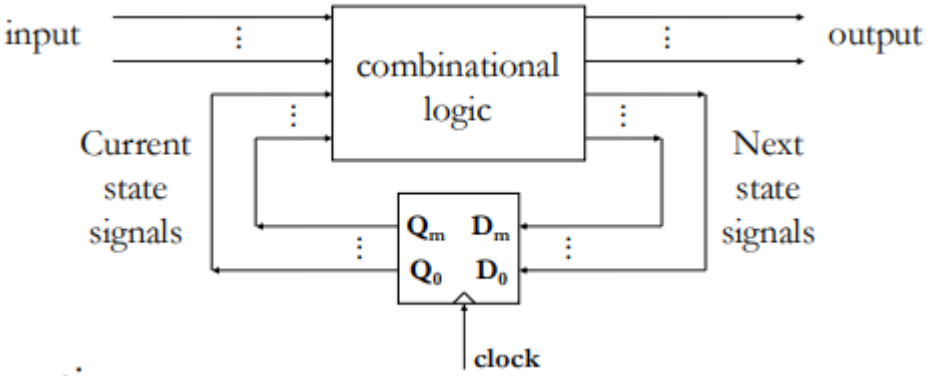
  - Only want to 'save' the data in a D latch when the data is ready
  - Set Q to D when Clock is 1; Ignore D if Clock is 0



- Using clock to build a memory element
  - Level triggered D latch: whenever clock is 1 D is propagated to Q

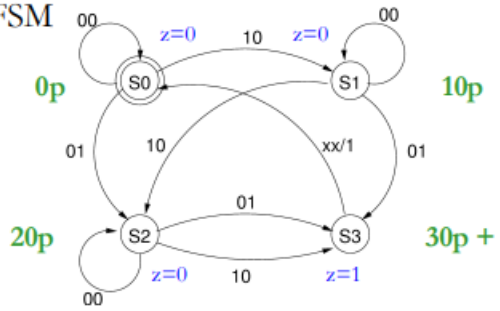
- Edge-triggered D latch: on a positive clock edge, D is propagated to Q (aka D flip-flop)
- 

- Register
  - Tie multiple D flip-flops together using a common clock
  - E.g., 4 bit register
  - 

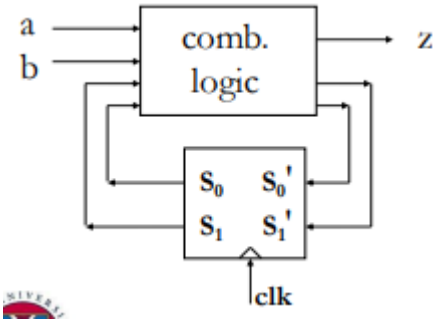
- Generational sequential logic circuit
  - 

  - Operation:
    - At every rising clock edge, next state signals are propagated to current state signals
    - Current state signal plus inputs work through combinational logic and generate output and next state signals
- Hardware FSM
  - A sequential circuit is a (deterministic) Finite State Machine – FSM
  - Example: Vending Machine

Accepts 10p, 20p coins, sells one product costing 30p, no change given
Coin reader has 2 signals: *a, b* for 10p, 20p coins respectively.
These are the inputs to the FSM
Output *z* asserted when 30p or more has been paid in



- FSM implementation
  - Methodology:
    - Choose encoding for states e.g. S0=00, …, S3=11
    - Build truth table for the next state s_1', s_0' and output z
    - Generate logic equations for s_1', s_0, z
    - Design comb logic from logic equations and add state holding register



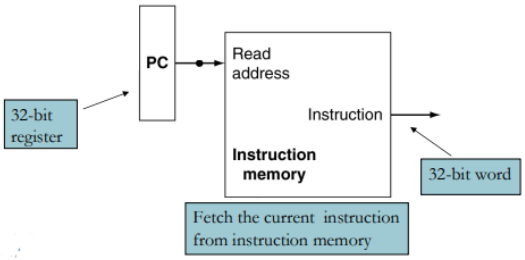
s_1	s_0	a	b	s_1'	s_0'	z
0	0	0	0	0	0	
0	0	0	1	1	0	0
0	0	1	0	0	1	
0	1	0	0	0	1	
0	1	0	1	1	1	1
0	1	1	0	1	0	

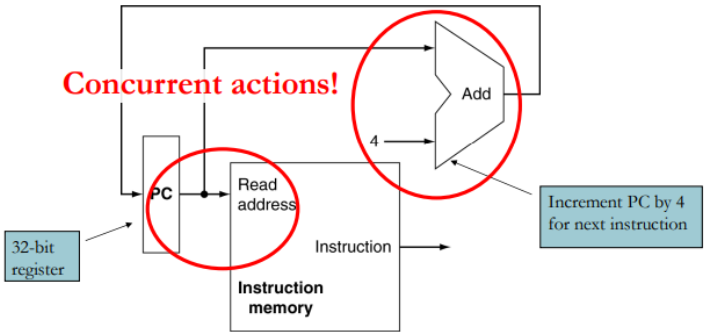
Processor Design – Single Cycle

- Two main parts:
  - Datapath: performs the data operation as commanded by the program instructions
  - Control: controls the datapath, memory and I/O according to the program instructions
  - Using:
    - Combinational and sequential circuits described in previous lectures
- Main processor functions

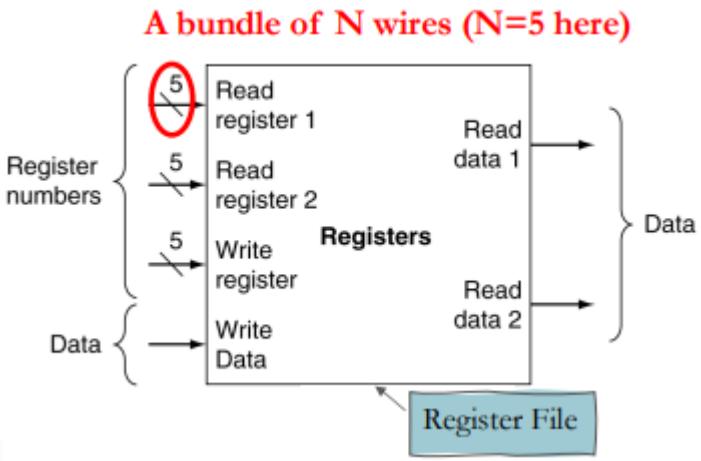
- Fetch instruction from instruction memory
- Read the register operands
- Use the ALU or computation
  - Arithmetic, memory address, branch target address
- Access data memory for load/store
- Store the result of computation or loaded data into the destination register
- Update the Program Counter (PC)

- Instruction Fetch: common to all instructions

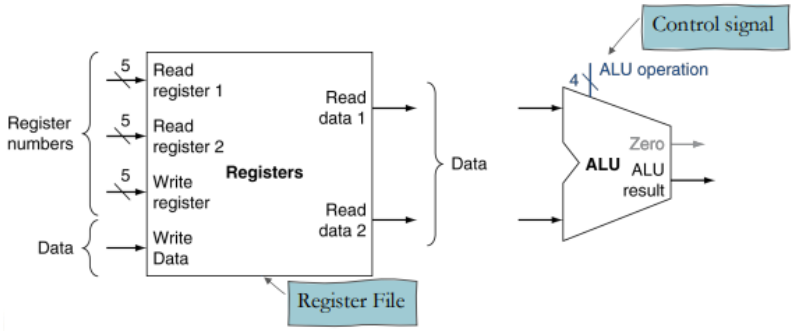




- R-Format Instructions
  - Read two register operands
  - Perform arithmetic/logical operation
  - Write register result
- Read two register Operands

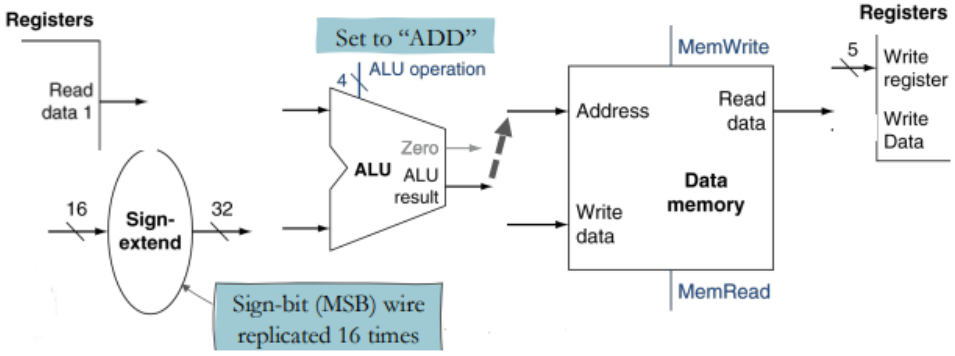


-
  - Perform arithmetic/logical operation
  - Write register result



-

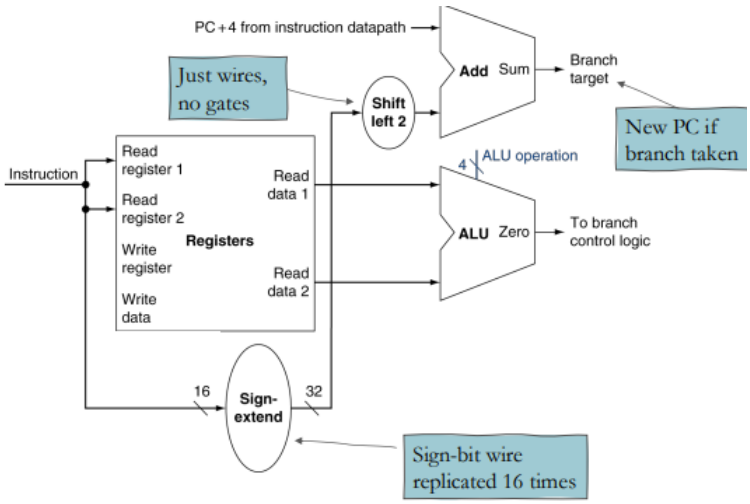
- Load/Store Instructions
  - Read register operands
  - Calculate address using a 16-offset
    - Use ALU, but first, sign-extend the offset
  - Read (for load) or write (for store) the memory
  - Load only: update destination register



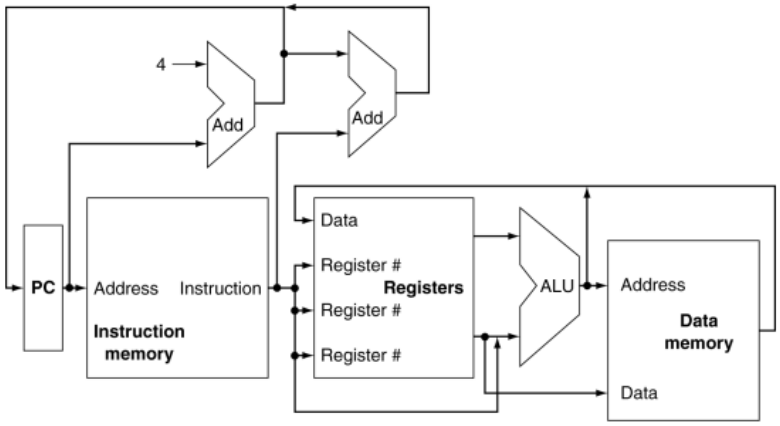
  -

- Branch Instructions
  - Read Register operands
  - Compare operands
    - Use ALU, subtract and check ALU's Zero output

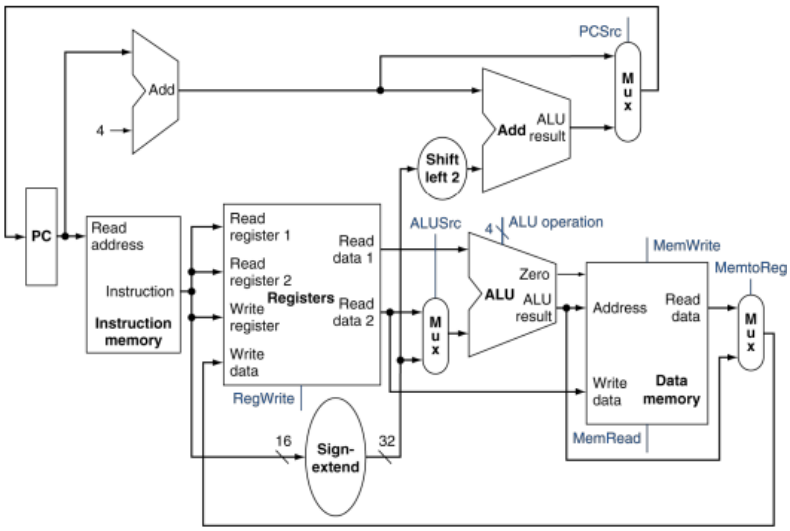
- Calculate target address
  - Sign-extend the immediate (offset)
  - Shift left 2 places (world align)
  - Add to PC + 4
  - Already calculated by instruction fetch



- Simplified Datapath



  - Can't just join wires together, multiplexers are used



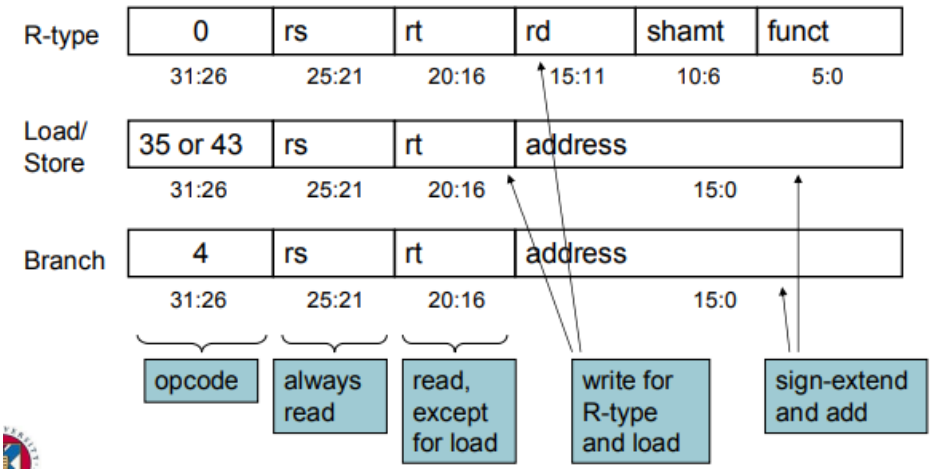
- Left – Instruction Fetch
- Bottom Right – R-type/Load/Store datapath
- Top Right- Branch Resolution
- Datapath attempts to execute all instructions in one clock cycle, therefore any datapath resource can only be used once per instruction.
- Multiplexor and control signal can be used to allow multiple connections to the input of an element

- Control Part
  - For all control signals determine which value selects what operation, input etc.
  - Make truth table of control signal values for each instruction or instruction group
  - Convert table to combinational circuit
  - Control signals derived from instruction fields

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

    - Depending on the instruction class, the ALU will need to perform one of these first five functions

- For LW and SW instructions we use the ALU to compute the memory address by addition
- For R-type instructions the ALU needs to perform one of the five actions, depending on the value of the 6-bit funct (or function) field in the low order bits of the instruction
- For branch equal, the ALU must perform a subtraction
- The 4 bit ALU control input can be generated using a small control unit that has as inputs the function field of the instruction and a 2 bit control field, called ALUOp. ALUOp indicates whether the operation to be performed should be add (00) for loads and stores, subtract (01) for beq, or determined by the operation in the funct field (10)
- The output of the ALU control unit is a 4 bit signal that directly controls the ALU by generating one of the 4 bit combinations seen previously



- The opcode determines the setting of the ALUOp bits
- The funct field is decoded by the ALU control design
- Shamt field is used only for shifts

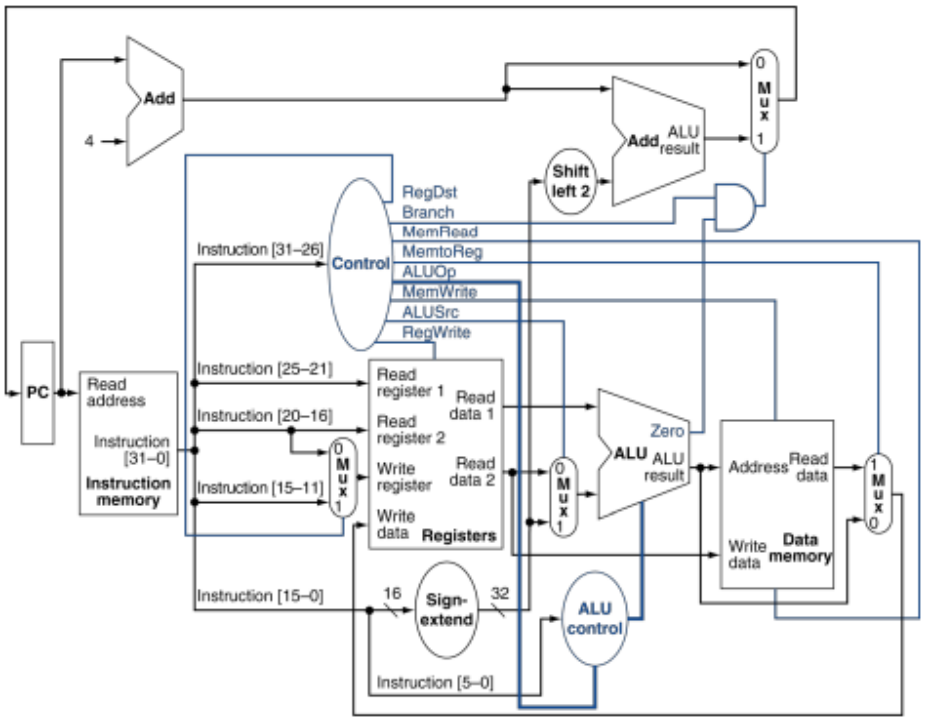
Instruction opcode	ALUOp	Instruction operation	Funct field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	0010
SW	00	store word	XXXXXX	add	0010
Branch equal	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
R-type	10	subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	set on less than	101010	set on less than	0111

- The x's are points where we don't care about what is in the field
- Using multiple levels of control reduces size of main control and can potentially increase the speed of the control unit

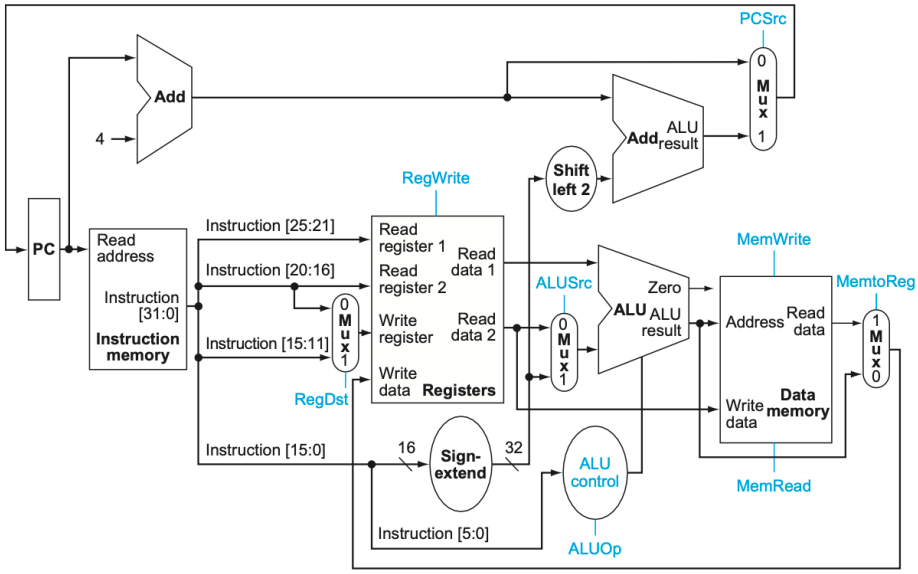
- Truth table shows how the 4 bit ALU control is set depending on these two input fields

ALUOp		Funct field						
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	Operation
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

-
-



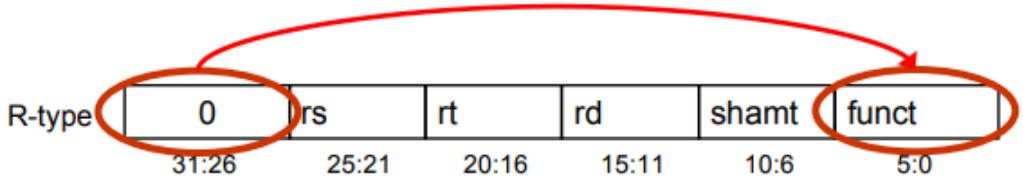
-
  - The instruction is split so that each portion goes to it's corresponding portion e.g 21-25 bits are read into register one because that's where the first register goes from instruction set



Signal name	Effect when deasserted	Effect when asserted
RegDst	The register destination number for the Write register comes from the rt field (bits 20:16).	The register destination number for the Write register comes from the rd field (bits 15:11).
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUOp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

- ALU operation:
  - Data transfers (ld/st) - add
  - Branches - sub
  - All other – determined by funct field, I[5:0]
  - These are all derived from the opcode



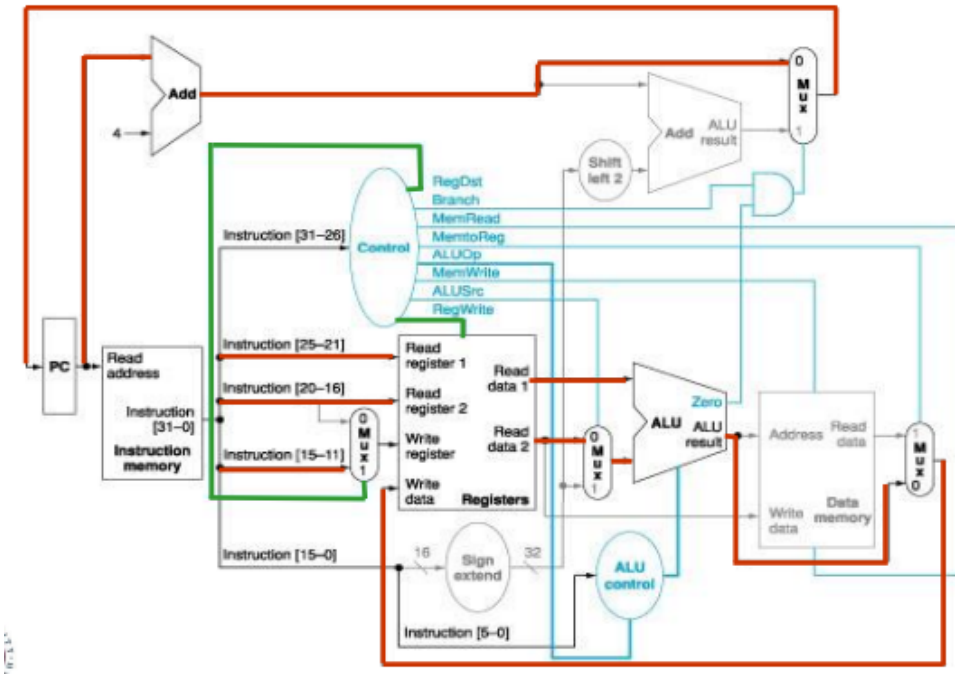
- ALU control is hierarchical
- Main control (ALUop) species which of the 3 op types
  - Add, Sub or based on Funct bits
- Second level provides actual ALU control signal

ALU operation	
0000	AND
0001	OR
0010	add
0110	subtract
0111	set-on-less than
...	

actual **ALU Control** signals

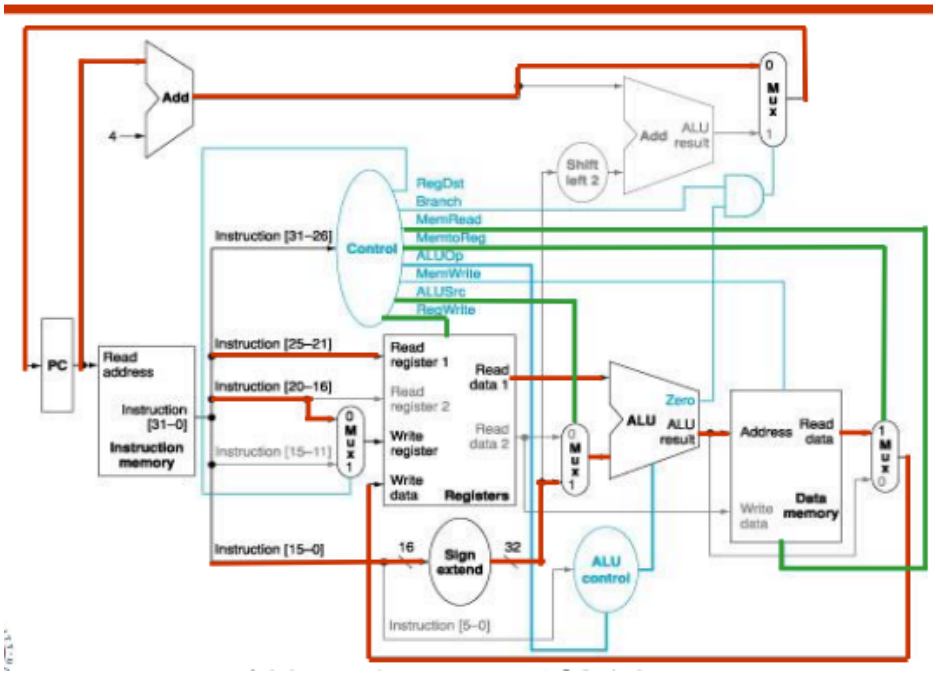
-

R-type instruction execution



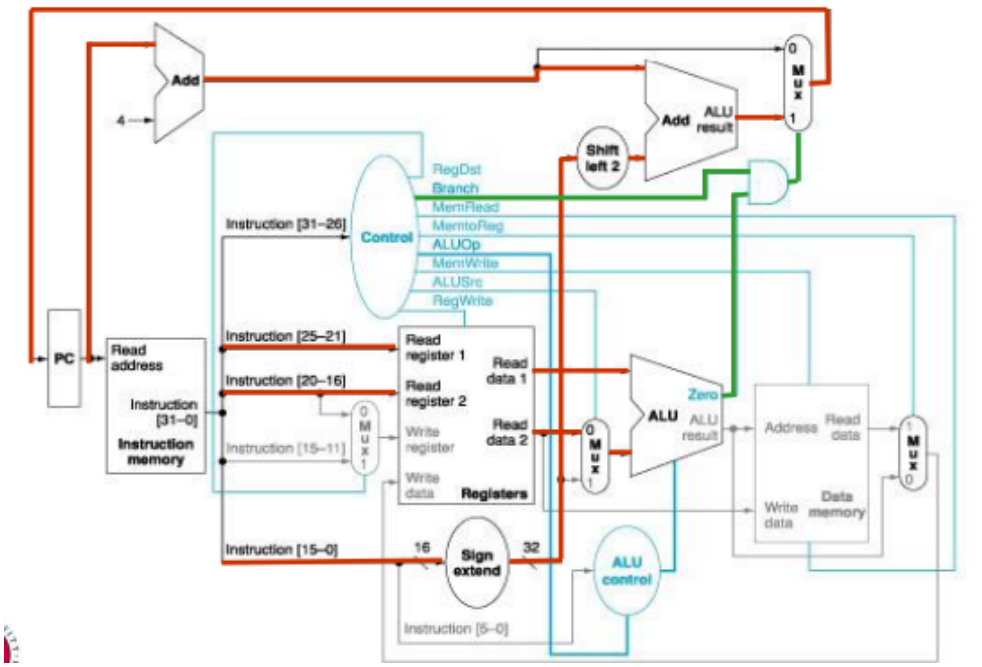
- The instruction is fetched and the PC is incremented
- Two registers are read from the register file, also the main control unit computes the setting of the control lines during this step
- The ALU operates on the data read from the register file, using the function code (bits 5:0, which is the funct field, of the instruction) to generate the ALU function
- The result from the ALU is written into the register file using bits 15:11 of the instruction to select the destination register

lw execution



-
  - An instruction is fetched from the instruction memory and the PC is incremented
  - A register value is read from the register file
  - The ALU computes the sum of the value read from the register and the sign extended, lower 16 bits of the instruction (offset)
  - The sum from the ALU is used as the address for the data memory
  - The data from the memory unit is written into the register file; the register destination is given by bits 20:16 of the instruction e.g. if the instruction is lw $t1 offset($t2)

beq (taken) execution

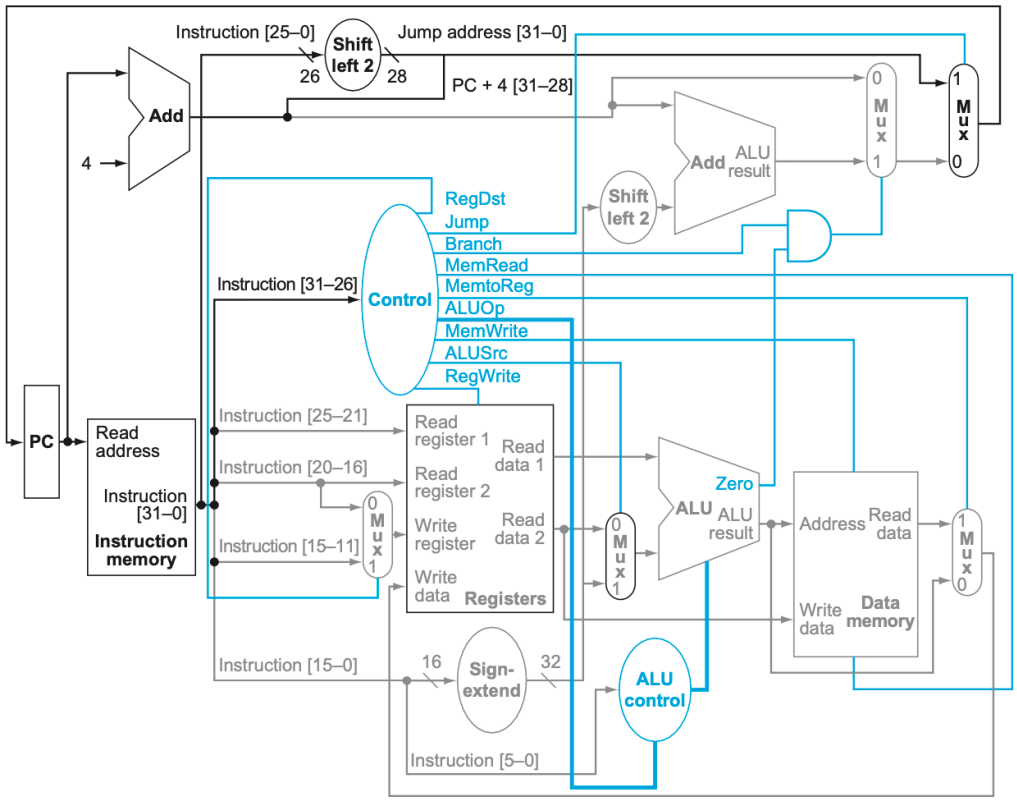


-
  - An instruction is fetched from the instruction memory, and the PC is incremented
  - Two registers are read from the register file
  - The ALU performs a subtract on the data values read from the register file. The value of PC + 4 is added to the sign-extended, lower 16 bits of the instruction (offset) shifted left by two; the result is the branch target address
  - The zero result from the ALU is used to decide which adder result to store into the PC

Input or output	Signal name	R-format	lw	sw	beq
Inputs	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
Outputs	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

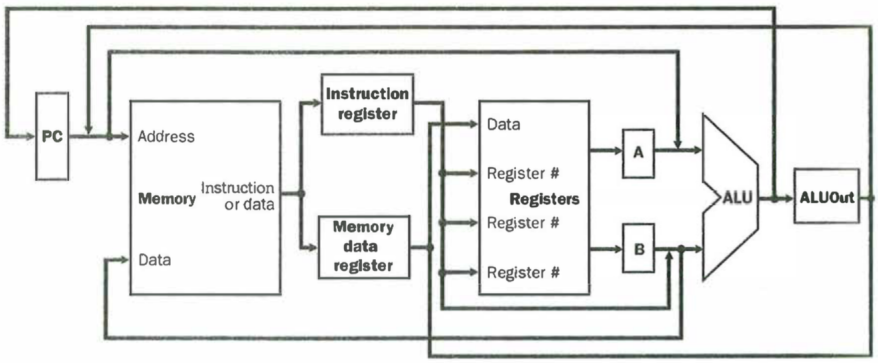
- Jump instruction

- | 000010 | address |
  - Address is at position 0:25
  - Lower 26 bits of the jump address are always 00
  - The next lower 26 bits of this 32 bit address come from the 26 bit immediate field in the instruction, the upper 4 bits of the address that should replace the PC come from the PC of the jump instruction plus 4.
  - Thus a jump can be implemented by storing into the PC a concatenation of
    - The upper 4 bits of the current PC + 4 (these are bits 31:28 of the sequentially following instruction address)
    - The 26 bit immediate field of the jump instruction
    - The bits 00
- Simple control and datapath extended to hand the jump instruction



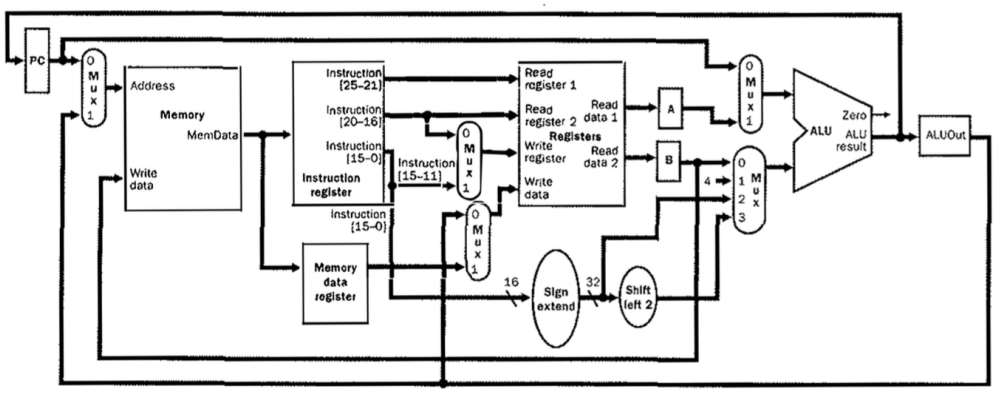
- The clock cycle in single-cycle design must have the same length for every instruction
- The longest possible path in the processor determines the clock cycle. This makes the common case slow.

Processor Design- Multi-Cycle

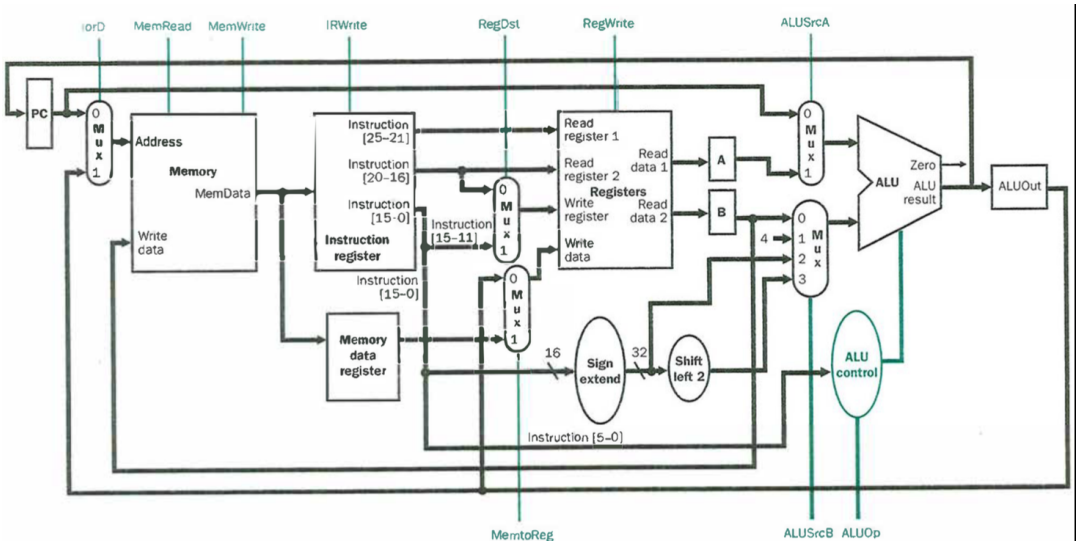
- Single Cycle Processors:
  - Speed: Cycle time must be long enough for the most complex instruction to complete
    - Average instruction needs less time
  - Cost: functional units (e.g. adders) cannot be re-used within one instruction's execution
- Execution time is : instruction count {programmer, compiler, ISA} * cycles per instruction * cycle time {Processor design}
- In a multicycle implementation each step in the execution will take 1 clock cycle. It allows a functional unit to be used more than once per instruction, as long as it used on different clock cycles
- Sharing reduces the amount of hardware required.
- Comparing this to the datapath for the single-cycle version
  - A single memory unit is used for both instructions and data
  - There is a single ALU, rather than an ALU and two adders
  - One or more registers are added after every major functional unit to hold the output of that unit until the value is used in a subsequent clock cycle
- High level view of the multicycle datapath:
  - 

  - At the end of a clock cycle, all data that are used in subsequent clock cycles must be stored in a state element.
  - Data used by subsequent instructions in a later clock cycle is stored into one of the programmer-visible state elements (i.e. the register file, the PC or the memory). Data used by the same instruction in a later cycle must be stored into one of these additional registers

- Temporary registers:
  - The Instruction register, the memory data register (MDR) are added to save the output of the memory for an instruction read and a data read, two separate registers are used, since, both values are needed during a clock cycle
  - The A and B registers are used to hold the register operand values read from the register file
  - The ALUOut register holds the output of the ALU
- All the registers except the IR hold data only between a pair of adjacent clock cycles and will thus not need a write control signal
- The IR needs to hold the instruction until the end of execution of that instruction and thus will require a write signal

- Replacing 3 ALUs of the single-cycle datapath by a single ALU means that the single ALU must accommodate all the inputs that used to go to 3. This requires two changes:
  - An additional multiplexor is added for the first ALU input. The multiplexor chooses between the A register and the PC
  - The multiplexor on the second ALU input is changed from a two-way multiplexor to a four0way multiplexor. The 2 additional inputs to the multiplexor are the constant 4 (for PC increment) and the sign-extended and shifted offset field (use in the branch address computation.
- Using registers and multiplexors (they are small), could reduce hardware cost
  - 

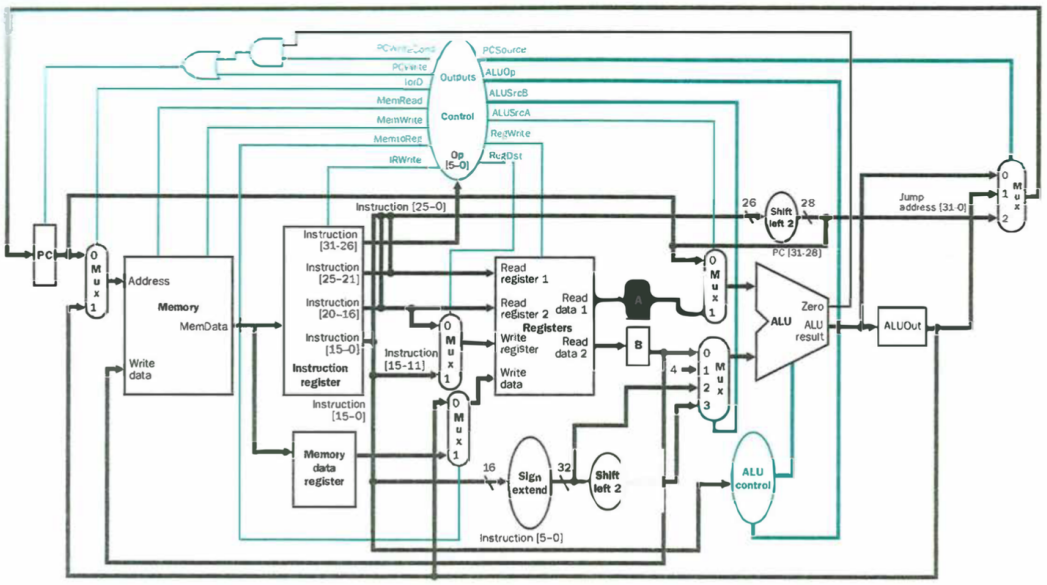
- The programmer-visible state units (the PC, the memory and the registers) as well as the IR will need write control signals. The memory will also need a read signal.

- The control unit can be used to control the new ALU here. Each of the two input multiplexors require a single control line, while the four-input multiplexor requires two control lines
  - 

  - The multicycle datapath, still requires additions to support branches and jumps
  - With the jump instruction and branch instruction there are 3 possible sources for the value to be written into the PC:
    - The output of the ALU, which is the value PC + 4 during the instruction fetch. This value should be stored directly into the PC
    - The register ALUOut, which is where the address of the branch target is stored
    - The lower 26 bits of the IR shifted by left by two and concatenated with the upper 4 bits of the incremented PC, which is the source when the instruction is a jump
  - The PC is written both unconditionally and conditionally. During a normal increment and jumps it is written unconditionally, however in branch the incremented PC is replaced with the value in ALUOut only if the two designated registers are equal. Thus the control needs two PC write signals, which we call PCWrite and PCWriteCond
  - A few gates are used to derive the PC write control signal from PCWrite, PCWriteCond and the Zero signal of the ALU, which is used to detect if two register operands of a beq are equal. To determine whether the PC should be written during a conditional

branch, we AND together the Zero signal of the ALU with the PCWriteCond

- The output of this AND gate is then Ored with PCWrite, which is the unconditional PC write signal. The output of this OR gate is connected to the write signal
- Complete datapath for the multicycle implementation + control:



- New additions: Multiplexor at top write to decide the source of a new PC value; two gates used to combine PC write signals (top left); and the control signals PCSource, PCWrite and PCWriteCond. The output of the IR is rearranged to send the lower 26 bits (the jump address) to the logic used to select the next PC. These 26 bits are shifted to the left by 2, adding 2 low-order 0 bits; these 28 bits are the concatenated with the high-order 4 bits of the PC, which has already been incremented

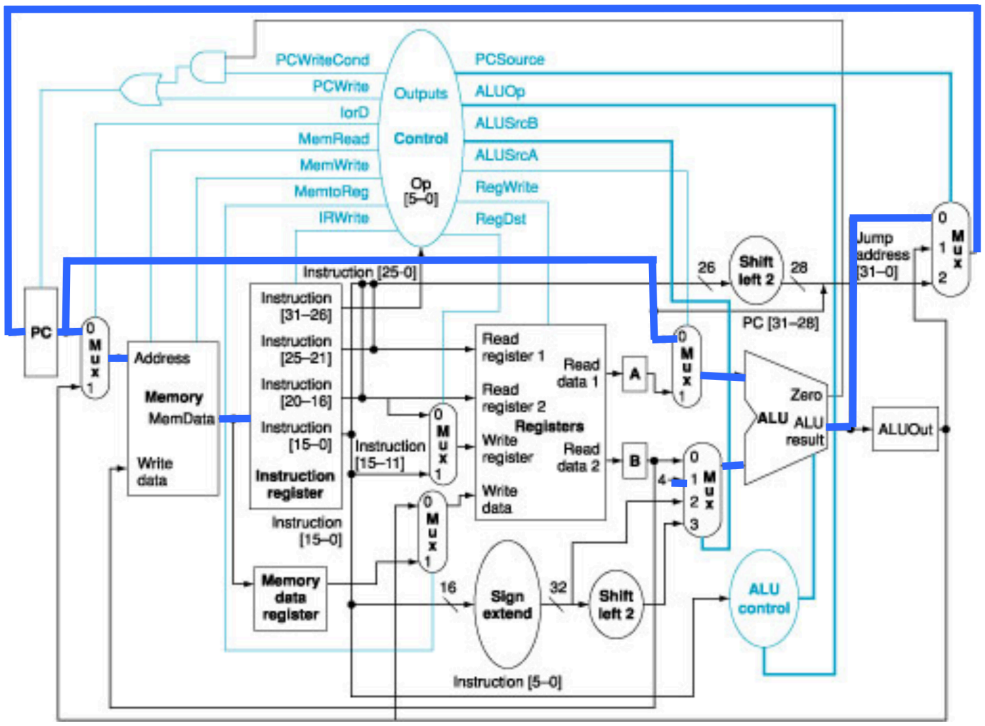
Actions of the 1-bit control signals

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register file destination number for the Write register comes from the rt field.	The register file destination number for the Write register comes from the rd field.
RegWrite	None	The general-purpose register selected by the Write register number is written with the value of the Write data input.
ALUSrcA	The first ALU operand is the PC.	The first ALU operand comes from the A register.
MemRead	None	Content of memory at the location specified by the Address input is put on Memory data output.
MemWrite	None	Memory contents at the location specified by the Address input is replaced by value on Write data input.
MemtoReg	The value fed to the register file Write data input comes from ALUOut.	The value fed to the register file Write data input comes from the MDR.
IorD	The PC is used to supply the address to the memory unit.	ALUOut is used to supply the address to the memory unit.
IRWrite	None	The output of the memory is written into the IR.
PCWrite	None	The PC is written; the source is controlled by PCSource.
PCWriteCond	None	The PC is written if the Zero output from the ALU is also active.

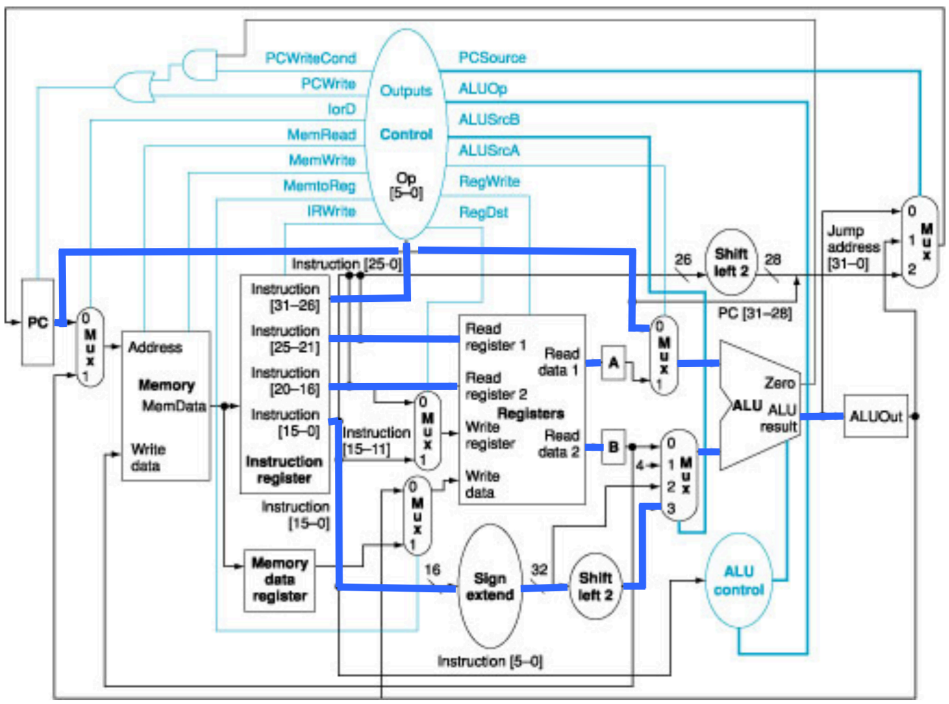
- **Actions of the 2-bit control signals**

Signal name	Value	Effect
ALUOp	00	The ALU performs an add operation.
	01	The ALU performs a subtract operation.
	10	The funct field of the instruction determines the ALU operation.
ALUSrcB	00	The second input to the ALU comes from the B register.
	01	The second input to the ALU is the constant 4.
	10	The second input to the ALU is the sign-extended, lower 16 bits of the IR.
	11	The second input to the ALU is the sign-extended, lower 16 bits of the IR shifted left 2 bits.
PCSource	00	Output of the ALU (PC + 4) is sent to the PC for writing.
	01	The contents of ALUOut (the branch target address) are sent to the PC for writing.
	10	The jump target address (IR[25–0] shifted left 2 bits and concatenated with PC + 4[31–28]) is sent to the PC for writing.

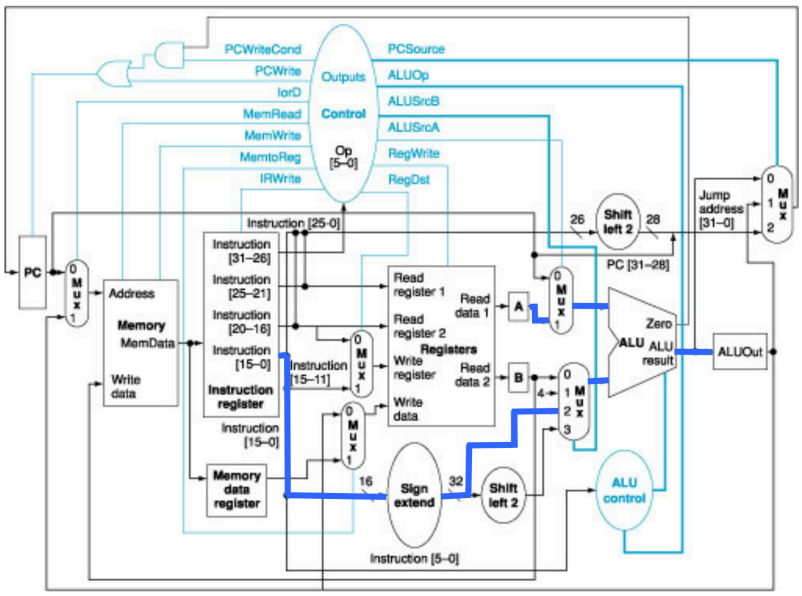
- Breaking the Instruction Execution into Clock Cycles
  - Each step takes exactly 1 clock cycle, only 1 ALU operation or one register file access or one memory access is allowed per cycle.
  - At the end of every clock cycle any data values that will be needed on subsequent cycle must be stored into a register, which can be either one of the major state elements (e.g., the PC, the register file, or the memory), a temporary register written on every clock cycle (e.g., A, B, MDR, or ALUOut), or a temporary register with write control (e.g., IR)
- Steps for execution
  1. Instruction fetch step
     - IR = Memory[PC]
     - PC = PC + 4
     - Send the PC to the memory as the address, perform a read and write the instruction into the IR, where it will stored.
     - Increment PC by 4
     - MemRead and IRWrite will need to be asserted, IorD = 0 to select PC as the source address. Incrementing PC by 4 requires ALUSrcA signal to 0 (sending the PC to ALU), ALUSrcB signal to 01 (sending 4 to the ALU) and ALUOp to 00 (to make the ALU add).
     - Storing the incremented instruction address back into PC requires setting PCWrite.
     - Increment of PC and the instruction memory access can occur in parallel.
     - New value of PC not visible until next clock cycle (incremented PC stored into ALUOut)

- 

2. Instruction decode and register fetch step
    - A = Reg[IR[25-21]]]; - IR[nn-nn] is index of that register
    - B = Reg[IR[20-16]];
    - ALUOut = PC + (sign-extend (IR[15-0]) << 2);
    - Actions can be performed without knowing the instruction if they apply to all instructions or if they are not harmful
    - Branch target address is computed, we can ignore the value if the instruction turns out not to be a branch
    - Performing optimistic actions early has the benefit of decreasing the number of clock cycles needed to execute an instruction
    - ALUSrcA is set to 0 (so that the PC is sent to the ALU), ALUSrcB to 11 (so that the sign-extended and shifted offset field is sent to the ALU), and ALUOp to 00 (so the ALU adds)
    - The register file accesses and computation of branch target occur in parallel

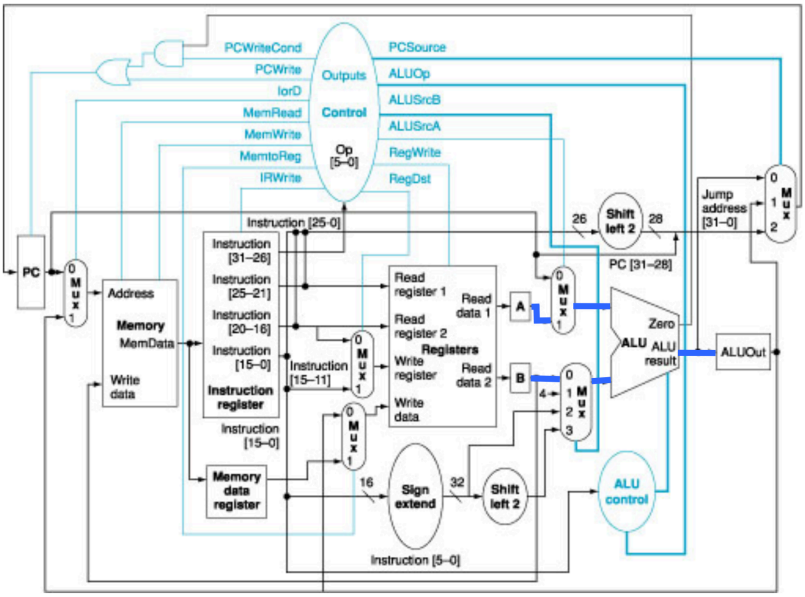


3. Execution Memory address computation or branch completion
    - First cycle datapath is determined by instruction class
        i. Memory Reference/ including immediate arithmetic
            - ALUOut = A + sign-extend (IR[15-0]);
            - The ALU is adding operands to from the memory address. ALUSrcA to 1 (so that the first ALU input is register A) and setting ALUSrcB to 10 (so that the output of the sign extension unit is used for the second ALU input). The ALUOp signals will be set to 00 (causing ALU to add)

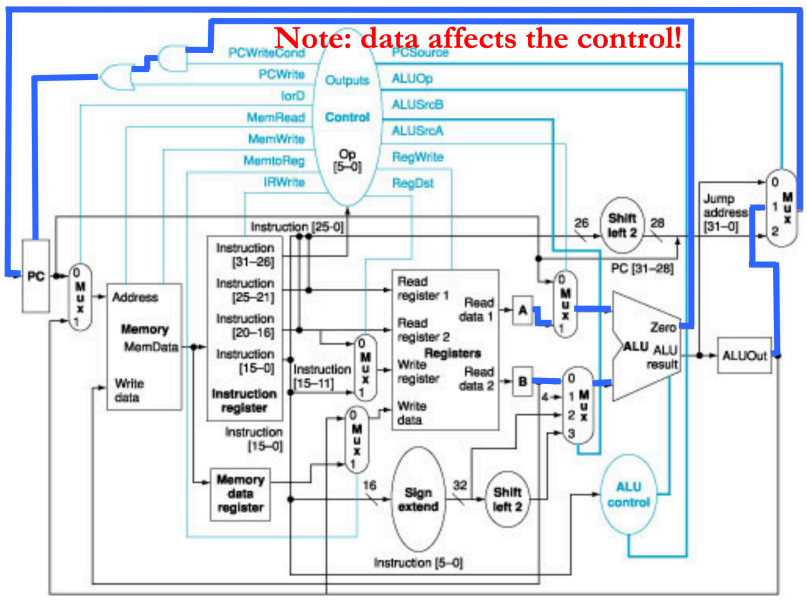


ii. Arithmetic-logical instruction (R-type)

- ALUOut = A op B;
- The ALU is performing the operation specified by the function code on the 2 values read from the register file in the previous cycle. This requires setting ALUSrcA = 1 and setting ALUSrcB = 00 (together causing the registers A and B to be used as the ALU inputs). The ALUOp signals will need to be set to 10 (so that funct field is used to determine the ALU control signal settings)

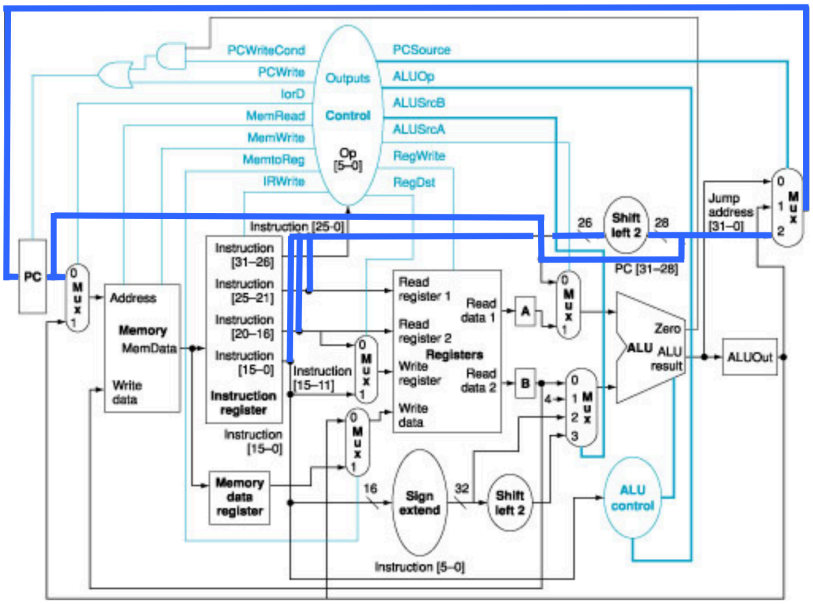


iii. Branch

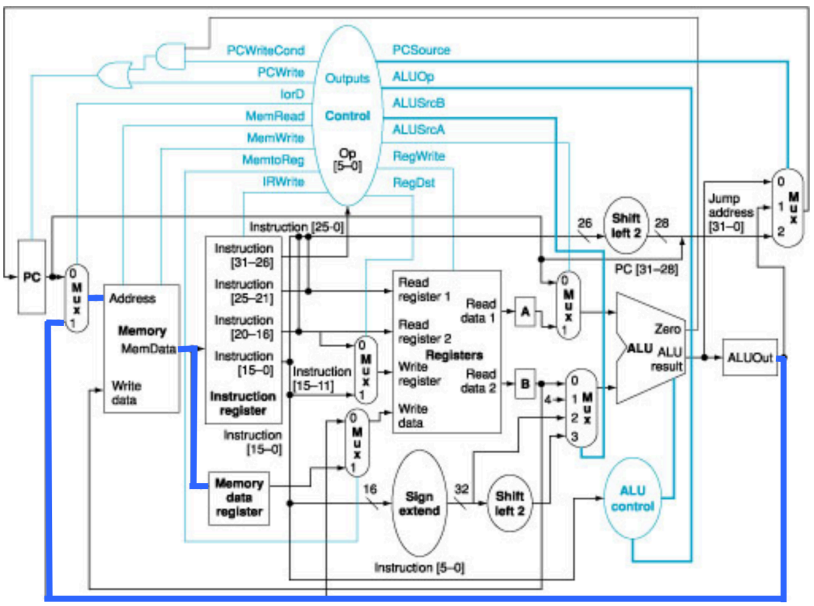
- If (A == B) PC = ALUOut;
- The ALU is used to do the equal comparison between the 2 registers read in the previous step. The Zero signal out of the ALU is used to determine whether or not to branch. ALUSrcA = 1 and setting ALUSrcB = 00 (so that the register file outputs are the ALU inputs)
- ALUOp will need to be set to 01 (subtraction)
- The PCCondWrite signal will need to be asserted to update the PC if the Zero output of the ALU is asserted. By setting PCSrouce to 01, the value written into the PC will come from ALUOut, which holds the branch target address computed in the previous cycle
- For conditional branches that are taken, we actually write the PC twice: once from the output of the ALU (during the Instruction decode/register fetch) and once from ALUOut (during the Branch completion step). The value written into the PC last is the one used for the next instruction fetch.
- 

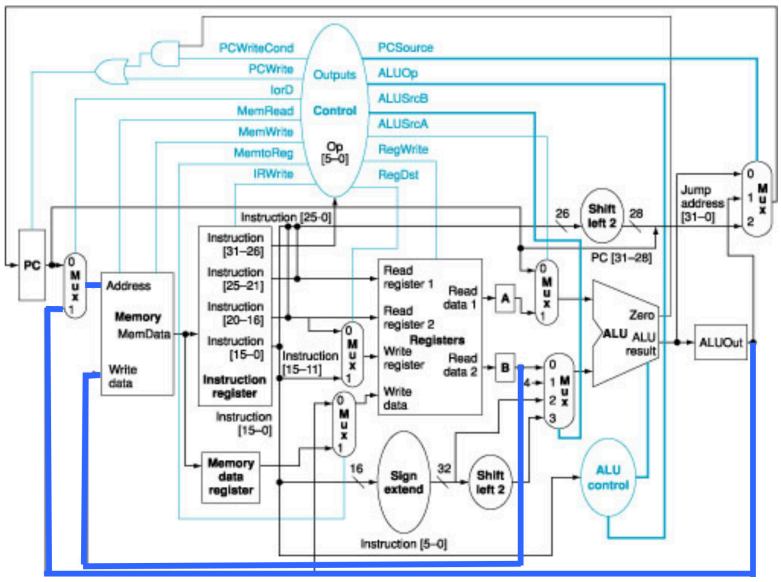
iv. Jump

- PC = PC [31-28] || (IR[25-0] << 2)

- The PC is replaced by jump address. PCSource is set to direct to the jump address to the PC and PCWrite is asserted to write the jump address into PC
- 

4. Memory Access or R-type instruction completion step
    i. Memory reference
        - MDR = Memory [ALUOut];
        - Memory [ALUOut] = B
        - If the instruction is a load, a data word is retrieved from memory and is written to the MDR
        - If the instruction is a store, then the data is written into memory.
        - In either case the address is computed during the previous step and stored in ALUOut.
        - For a store, the source operand is saved in B (B is actually read twice, once in step 2 and once in step 3). The same value is read both times, since the register number – which is stored in IR and used to read from the register file – does not change).
        - Signal MemRead (for a load) or MemWrite (fro store) will need to be asserted

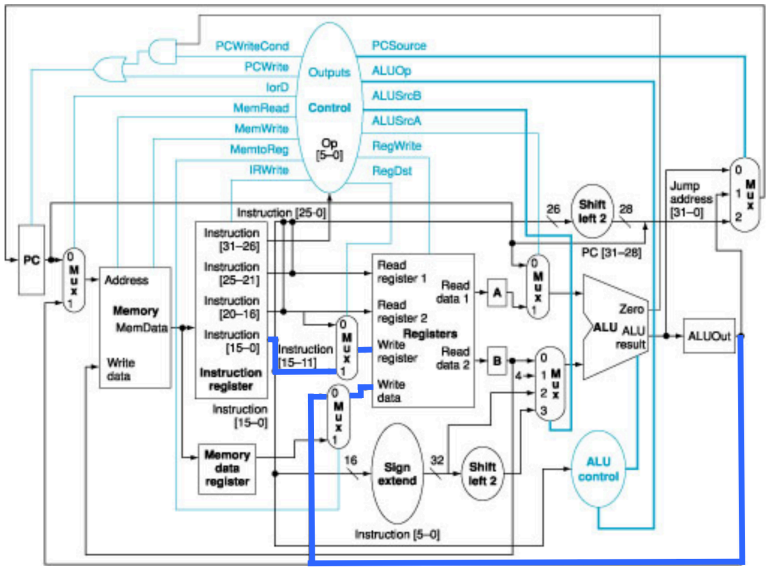
- IorD is set to 1 to force the memory address to come from the ALU, rather than the PC. Since MDR is written on every clock cycle, no explicit control need be asserted
- Load:
- 

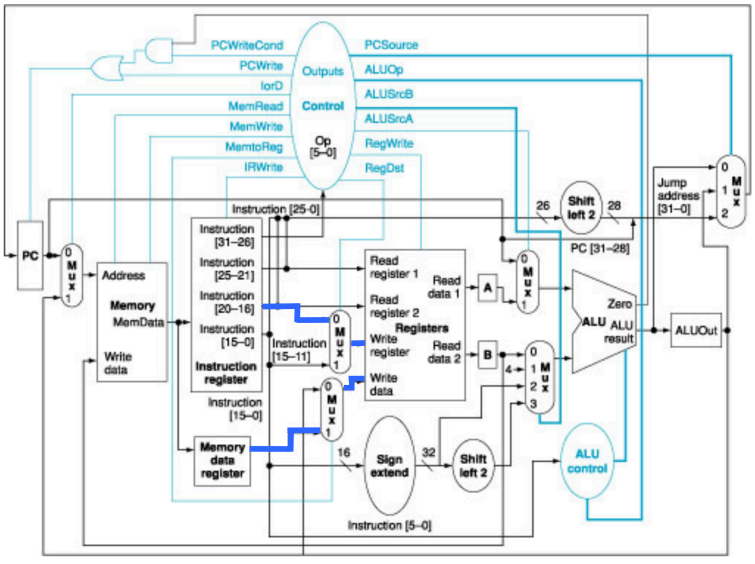
- Write:
- 

ii. R type Instructions

- Reg[IR[15-11]] = ALUOut;
- Place contents of ALU, which corresponds to the output of the ALU operation in the previous cycle, into the result register
- Signal RegDst must be set to 1 (to focrce rd (bits 15-11) field to be used to select the

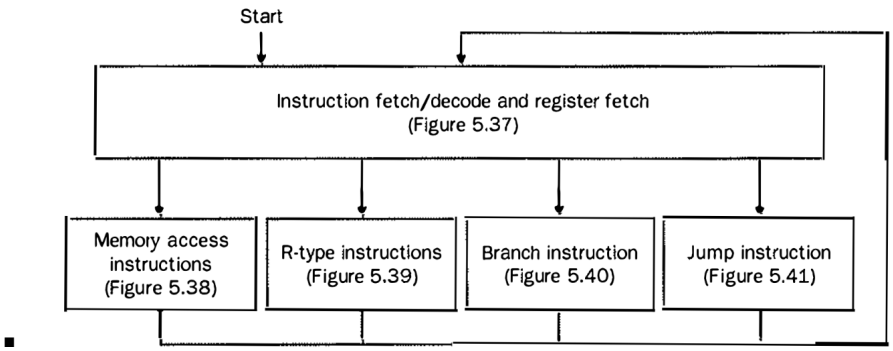
register file entry to write). RegWrite must be asserted and MemtoReg must be set to 0 (so that the output ALU is written as opposed to the memory data output

- 

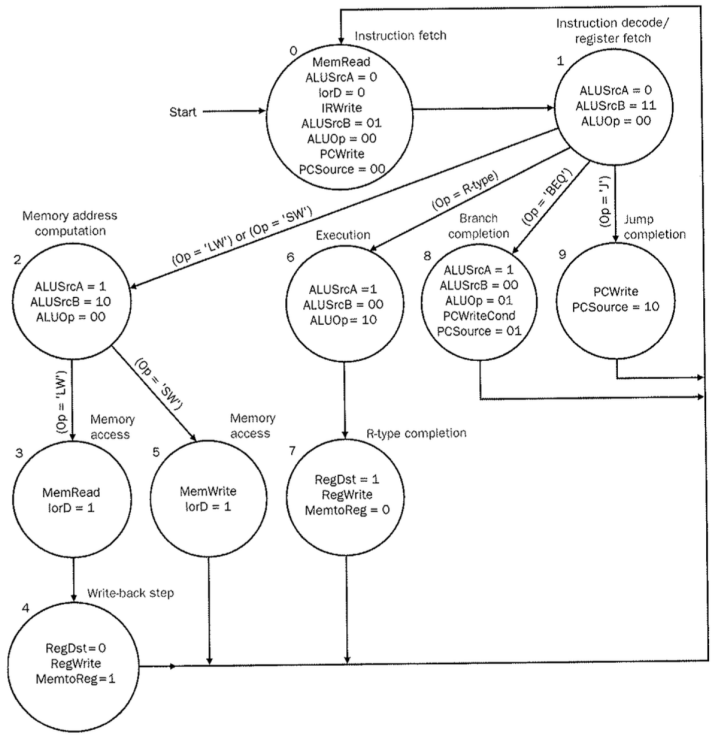
5. Memory read completion step
    i. Memory (Load)
        - Reg[IR[20-16]] = MDR;
        - Write the load data, which was stored into MDR in the previous cycle, into the register file
        - MemtoRed = 1 (to write the result from memory), assert RegWrite (to cause a write) and make RegDst = 0 to choose the rt (bits 20-16) field as the register number
        - 

Step name	Action for R-type instructions	Action for memory-reference instructions	Action for branches	Action for jumps
Instruction fetch	IR = Memory[PC] PC = PC + 4			
Instruction decode/register fetch	A = Reg [IR[25–21]] B = Reg [IR[20–16]] ALUOut = PC + (sign-extend (IR[15–0]) << 2)			
Execution, address computation, branch/ jump completion	ALUOut = A op B	ALUOut = A + sign-extend (IR[15–0])	if (A == B) then PC = ALUOut	PC = PC [31–28] || (IR[25–0]<<2)
Memory access or R-type completion	Reg [IR[15–11]] = ALUOut	Load: MDR = Memory[ALUOut] or Store: Memory [ALUOut] = B		
Memory read completion		Load: Reg[IR[20–16]] = MDR		

- Control
  - The control for a multicycle datapath can be an FSM (finite state machine) or microprogramming
  - FSM consists of a set of states and directions on how to change states. The directions are defined by a next state function, which maps current state and the inputs to a new state
  - The implementation of a FSM assumes all outputs that are not explicitly asserted are deasserted. Only values of interest are specified in FSM
  - High Level view of the finite state machine



  - Complete finite state machine control for the datapath



CPI in a Multicycle CPU

Using the control shown in Figure 5.42 and the gcc instruction mix shown in the example starting on page 373, what is the CPI, assuming that each state requires 1 clock cycle?

The mix is 22% loads, 11% stores, 49% R-format operations, 16% branches, and 2% jumps. From Figure 5.42, the number of clock cycles for each instruction class is the following:

- Loads: 5
- Stores: 4
- R-format instructions: 4
- Branches: 3
- Jumps: 3

The CPI is given by the following:

$$\text{CPI} = \frac{\text{CPU clock cycles}}{\text{Instruction count}} = \frac{\sum \text{Instruction count}_i \times \text{CPI}_i}{\text{Instruction count}}$$

$$= \sum \frac{\text{Instruction count}_i}{\text{Instruction count}} \times \text{CPI}_i$$

The ratio

$$\frac{\text{Instruction count}_i}{\text{Instruction count}}$$

is simply the instruction frequency for the instruction class i. We can therefore substitute to obtain

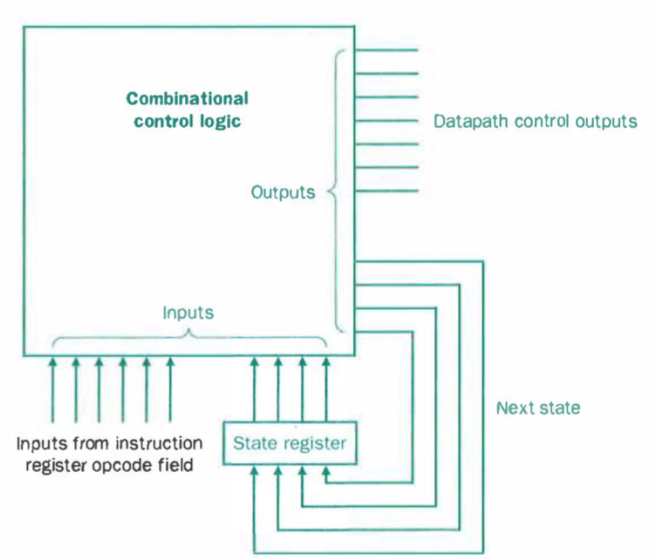
$$\text{CPI} = 0.22 \times 5 + 0.11 \times 4 + 0.49 \times 4 + 0.16 \times 3 + 0.02 \times 3$$

$$= 1.1 + 0.44 + 1.96 + 0.48 + 0.06 = 4.04$$

This CPI is better than the worst-case CPI would have been if all the instructions took the same number of clock cycles (5).

- CPI = Cycles per instruction

- A finite state machine can be implemented with a temporary register that holds the current state and a block of combinational logic that determines both the datapath signals to be asserted as well as the next state

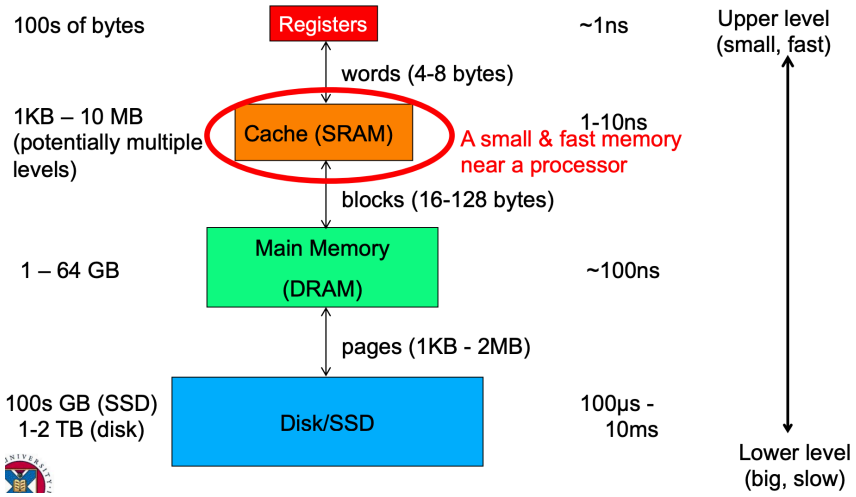


-
- The outputs of the combinational logic are the next state number and the control signals to be asserted for the current state.
- The inputs to the combinational logic are the current state and any inputs used to determine the next state. The inputs are the instruction register opcode bits
- Execution time. = instruction count * cycles per instruction * cycle time

Memory Hierarchy and Caches

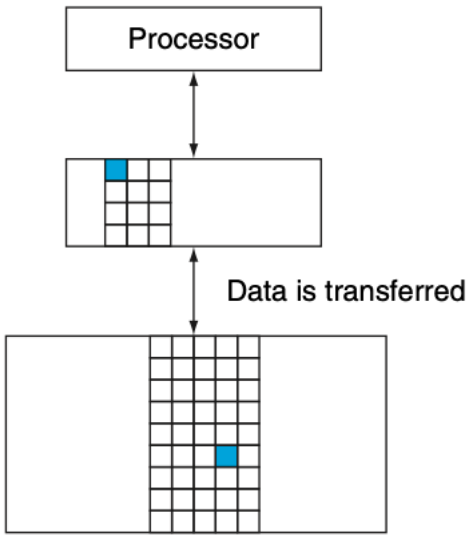
- Overview – using a combination of memory kinds
    - Smaller amounts of expensive but fast memory closer to the processor
    - Larger amounts of cheaper but slower memory farther from the processor
    -

Technology	Typical access time	Price per GB
SRAM	1-10 ns	£1000
DRAM	~100 ns	£10
Flash SSD	~100 μs	£1
Magnetic disk	~10 ms	£0.1



- DRAM is the main memory

- Temporal locality (locality in time): if an item is referenced, it will tend to be referenced again soon
- Spatial locality: if an item is referenced, items whose addresses are close by will tend to be referenced soon
- Memory hierarchy – A structure that uses multiple levels oof memories; as the distance from the processor increases, the size of the memories and access time both increase
- Data is copied between only 2 adjacent levels at a time, so we can focus our attention on just two levels
- The upper level- the one closer to the processor – is smaller and faster than the lower level, since the upper level uses technology that is more expensive

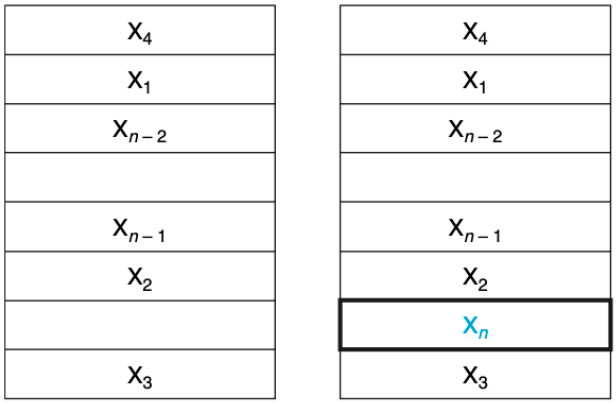


- A block or line (in blue) is the minimum unit of information that can be present or not in a cache

    - If data requested by the processor appears in some block in the upper level this is called a hit. If the data is not found in the upper level, the request is called a miss. The lower level in the hierarchy is then accessed to retrieve the block containing the requested data.
    - The hit rate or hit ration is the fraction of memory accesses found in a level of memory
    - The miss rate is the fraction of memory accesses not found in a level of the memory hierarchy
    - Hit time is the time to access the upper level of the memory hierarchy, which includes the time needed to determine whether the access is a hit or a miss
    - The miss penalty is the time required to fetch a block into a level of memory from the lower level, including the time to access the block, transmit it from one level to the other, insert it in the level that experienced the miss and then the pass the block to the requestor
- Cache – a small fast memory near a processor
- Small fast cache backed up by larger slower cache(s) and main memory gives the impression of a single fast memory
- Take advantage of temporal locality
    - If access data is in slower memory, move it to faster memory
    - If data in faster memory is unused recently move it to slower memory
- Take advantage of spatial locality
    - If need to move a word from slower to faster memory, move adjacent words at same time
    - Gives rise to blocks and pages: units of storage within the memory hierarchy composed of multiple contiguous words
- Handling of memory hierarchy by software and hardware
    - *SW (compiler):* between registers and cache/main memory
    - *HW:* between caches and main memory (SW is usually unaware of caches)
    - *SW (Operating System):* between main memory and disk/SSD
- HW managed transfers between levels

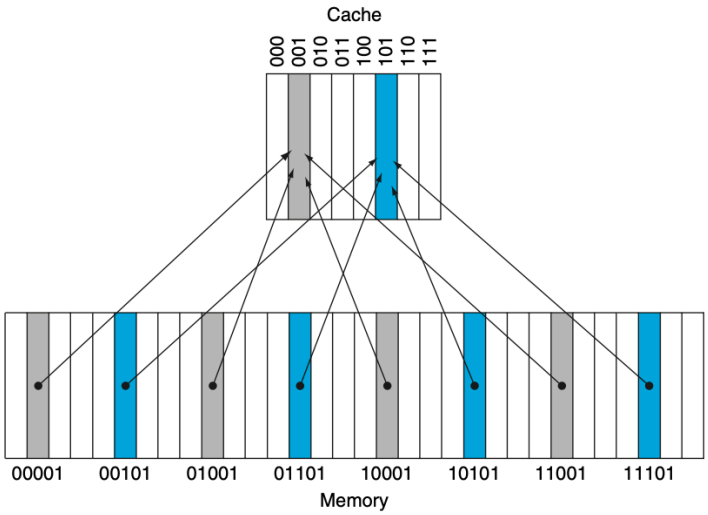
- Occurs between cache and main memory levels
- Programmer and processor oblivious to where data resides
- Cache hardware manages transfers
- Memory always has a copy of cached data, but in data in the cache may be more recent

- Cache basics



a. Before the reference to X_n b. After the reference to X_n

- X_n is initially not in the cache. The processor request a word X_n that is not in cache. This results in a miss and word X_n is brought from memory into cache.
- The simplest way to assign cache for each word is to assign the cache location based on the address of the word in memory.
- Direct-mapped cache - A structure in which each memory location is mapped to exactly one location in the cache i.e Cache location = (Block address) % (Number of blocks in the cache)
- If the number of entries in the cache is a power of 2, then modulo can be computed by using the low-order $\log_2$ (cache size in blocks) bits of the address. Thus an 8-block cache uses the three lowest bits ($8 = 2^3$) of the block address



- We use tags to know whether the requested word is in the cache.
- Tag – a field in a table used for a memory hierarchy that contains the address information required to identify whether the associated block in the hierarchy corresponds to a requested word
- The tags contain the address information required to identify whether a word in cache corresponds to the requested word. The tag only neds to contain the upper portion of the address, corresponding to the bits that are not used as an index into the cache.
- In the diagram above, only upper 2 of the 5 address bits in the tag, since the lower 3-bit index field of the address selects the block
- The index bits because they are omitted because they are redundant, since by definition the index field of any address of a cache block must be that block number
- We also need a way to recognize that a cache block does not have valid information. Thus a valid bit is needed.
- A valid bit field in the tables of a memory hierarchy indicates the associated block in the hierarchy contains valid data

- Accessing a Cache
  - Check index; Check valid bit (filled with something); Check tag to see if matches; if match hit, if no match miss and overwrite(temporal locality)
  - Example

-

Decimal address of reference	Binary address of reference	Hit or miss in cache	Assigned cache block (where found or placed)
22	10110_{two}	miss (5.6b)	$(10110_{two} \bmod 8) = 110_{two}$
26	11010_{two}	miss (5.6c)	$(11010_{two} \bmod 8) = 010_{two}$
22	10110_{two}	hit	$(10110_{two} \bmod 8) = 110_{two}$
26	11010_{two}	hit	$(11010_{two} \bmod 8) = 010_{two}$
16	10000_{two}	miss (5.6d)	$(10000_{two} \bmod 8) = 000_{two}$
3	00011_{two}	miss (5.6e)	$(00011_{two} \bmod 8) = 011_{two}$
16	10000_{two}	hit	$(10000_{two} \bmod 8) = 000_{two}$
18	10010_{two}	miss (5.6f)	$(10010_{two} \bmod 8) = 010_{two}$
16	10000_{two}	hit	$(10000_{two} \bmod 8) = 000_{two}$

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

a. The initial state of the cache after power-on

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

b. After handling a miss of address (10110_{two})

Index	V	Tag	Data
000	N		
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

c. After handling a miss of address (11010_{two})

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

d. After handling a miss of address (10000_{two})

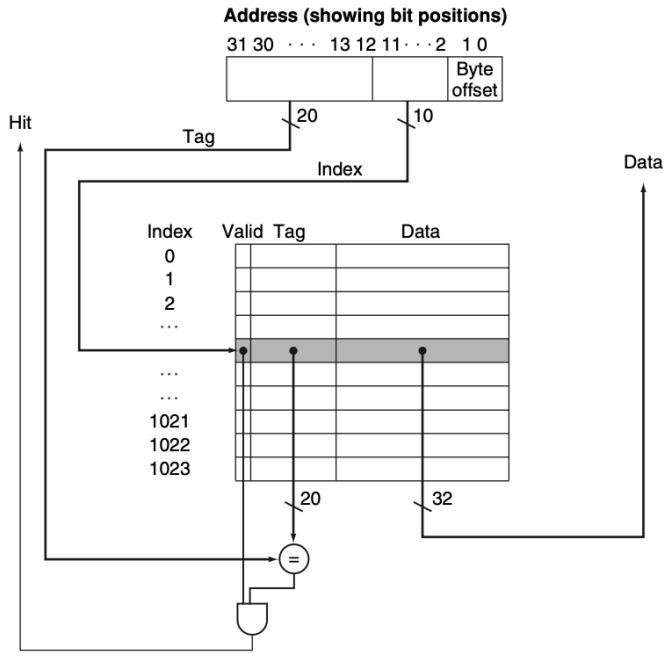
Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

e. After handling a miss of address (00011_{two})

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	10_{two}	Memory (10010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

f. After handling a miss of address (10010_{two})

- When address 10010_{two} (18) is referenced, the entry address 11010_{two} (26) must be replaced and a reference to $1101o_{two}$ will cause a subsequent miss. The tag field will contain only the upper portion of the address. The full address of a word contained in a cache block i with tag field j for cache is j * 8 + i or equivalently the concatenation of the tag field j and the index i. E.g. in cach f above, index 010_{two} has tag 10_{two} and corresponds to address 10010_{two}



-
  - Cache holds 4KB
  - Tag from the cache is compared against the upper portion of the address to determine whether the entry in the cache corresponds to the requested address
- A tag field which is used to compare with the value of the tag field of the cache
- A cache index, which is used to select the block
- The index of a cache block together with the tag contents of that block uniquely specifies the memory address of the word contained in the cache block
- The total number of entries in a direct-mapped cache must be a power of 2
- The total number of bits needed for a cache is a function of the cache size and the address size because the cache includes both storage for the data and the tags
- For the following situation: 32-bit address; a direct-mapped cache; the cache size is 2^n blocks, so n bits are used for the index
- The block size 2^m words (2^{m+2} bytes), so m bits are used for the word within the block, and two bits are used for the byte part of the address

- Size of tag field is 32 – (n + m + 2)
- The total number of bits in a direct-mapped cache is 2^n * (block size + tag size + valid field size)

- Examples
  - **Bits in a Cache**

    How many total bits are required for a direct-mapped cache with 16 KiB of data and 4-word blocks, assuming a 32-bit address?

    We know that 16 KiB is 4096 (2^{12}) words. With a block size of 4 words (2^2), there are 1024 (2^{10}) blocks. Each block has 4×32 or 128 bits of data plus a tag, which is $32 - 10 - 2 - 2$ bits, plus a valid bit. Thus, the total cache size is

    $$2^{10} \times (4 \times 32 + (32 - 10 - 2 - 2) + 1) = 2^{10} \times 147 = 147 \text{ Kibibits}$$

    or 18.4 KiB for a 16 KiB cache. For this cache, the total number of bits in the cache is about 1.15 times as many as needed just for the storage of the data.
  - **Mapping an Address to a Multiword Cache Block**

    Consider a cache with 64 blocks and a block size of 16 bytes. To what block number does byte address 1200 map?

    We saw the formula on page 384. The block is given by

    (Block address) modulo (Number of blocks in the cache)

    where the address of the block is

    $$\frac{\text{Byte address}}{\text{Bytes per block}}$$

    Notice that this block address is the block containing all addresses between

    $$\left\lfloor \frac{\text{Byte address}}{\text{Bytes per block}} \right\rfloor \times \text{Bytes per block}$$

    and

    $$\left\lfloor \frac{\text{Byte address}}{\text{Bytes per block}} \right\rfloor \times \text{Bytes per block} + (\text{Bytes per block} - 1)$$

    Thus, with 16 bytes per block, byte address 1200 is block address

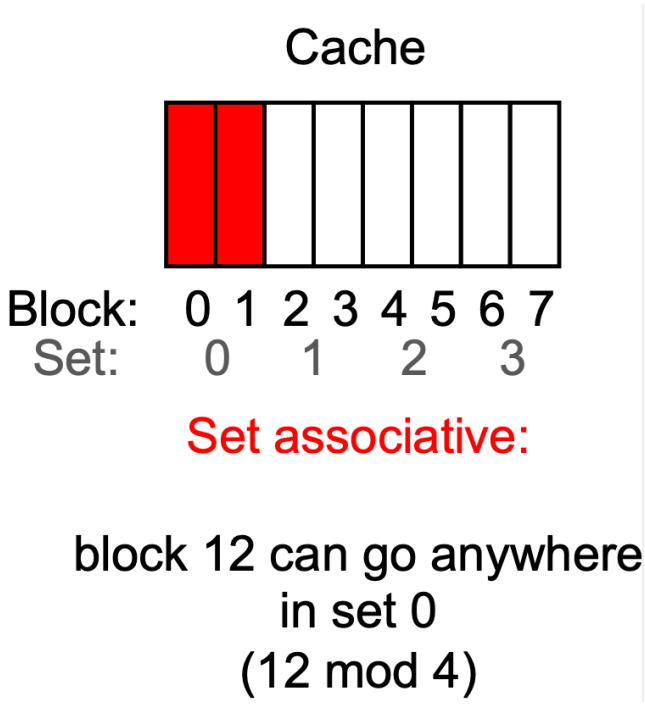
    $$\left\lfloor \frac{1200}{6} \right\rfloor = 75$$

    which maps to cache block number (75 modulo 64) = 11. In fact, this block maps all addresses between 1200 and 1215.
  - Larger blocks exploit spatial locality to lower miss rates
  - Miss rate mya go up eventually if the block size becomes a significant fraction of the cache size, because the number of

blocks that can be held in the cache will become small, and competition for those blocks increases

- Handling cache misses
    - A cache miss is a request for data from the cache that cannot be filled because the data is not present in the cache
    - Processing of a cache miss creates a pipeline stall as opposed to an interrupt, which would require saving the state of all registers.
    - For a cache miss, we can stall the entire processor, essentially freezing the contents of the temporary and programmer-visible registers, while we wait for memory.
    - Steps taken on the instruction cache miss:
        - Send the original PC value (current PC – 4) to the memory
        - Instruct main memory to perform a read and wait for the memory to complete its access
        - Write the cache entry, putting data from memory in the data portion of the entry, writing the upper bits of address (from the ALU) into the tag field and turning the valid bit on
        - Restart the instruction execution at the first step, while will refetch the instruction, this time finding it in the cache
- Handling Writes
    - Cache and memory are inconsistent when write happens on the data cache without changing main memory
    - The simplest way to keep the main memory and cache consistent is always to write the data into both the memory and the cache. This is called write through
    - Write-through – A scheme in which writes always update both the cache and the next lower level of the memory hierarchy, ensuring that data is always consistent between the two
    - During a write miss, we first fetch the words of the block from memory; the block is fetched and placed into the cache block; Wite the word to main memory using the full address
    - This will be slow, solution is write buffer.
    - A write buffer is a queue that holds data while the data is waiting to be written to memory A write buffer stores the data while it is waiting to be written to memory

    - After writing the data into the cache and into the write buffer, the processor can continue execution. When a write to main memory completes, the entry in the write buffer is freed. If the write buffer is full the processor must stall until there is an empty position in the write buffer.
    - If the rate at which the memory can complete writes is less than the rate at which the processor is generating writes, no amount of buffering can help, because writes are being generated faster than the memory system can accept them.
    - Alternative to write through scheme is called write-back
    - Write-back is a scheme that handles writes by updating values only to the block in the cache, then writing the modified block to the lower level of the hierarchy when the block is replaced
        - Good: Writes are fast and generate little memory traffic
        - Bad: Memory can have stale data for some time
- Fully associative cache only uses a tag. A block can be placed anywhere. Correct cache block identified by matching tags
- Cache replacement: Least Recently Used – Evict the cache block that hasn't been accessed longest, relies on past behaviour as a predictor of the future; Fifo – replace in same order as filled (simple to implement)
- Set Associative
    - Split the cache into groups (sets) of m blocks each -> m-way set-associative
    - A given block can go only into one set (based on block address), but within that set it can go anywhere
    - Good compromise between direct-mapped and fully-associative caches
    - Typical degree of associativity is 2-16
    - Example of set associative cache:



- Write allocate – bring block into the cache and write to it; useful if locality exits
- Write no-allocate – do not bring the block into the cache; modify data only in memory; useful if no locality, guarantees cache and memory have the same value address)

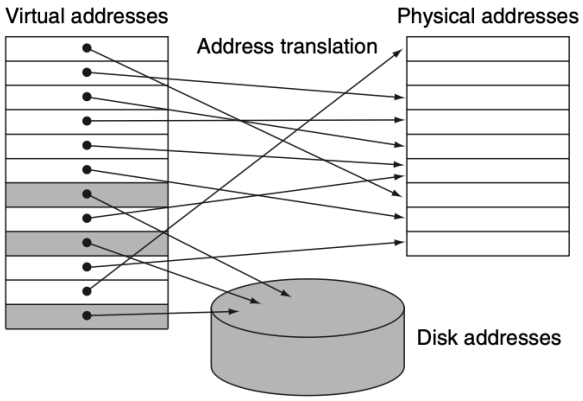
	Fully Associative	**Direct Mapped**	**2-way Set Associative**	**4-way Set Associative**
# of blocks	1KB/4 bytes = 256	1KB/4 bytes = 256	1KB/4 bytes = 256	1KB/4 bytes = 256
Offset	4 bytes = 2 bits	4 bytes = 2 bits	4 bytes = 2 bits	4 bytes = 2 bits
Index	N/A	256 index; 8 bits	256/2 sets; 7 bits	256/4 sets; 6 bits
Tag	30 bits (32 -2)	22 bits (32 – 2 – 8)	23 bits (32 – 2 – 7)	24 bits (32 – 2 – 6)

Virtual Memory

- Virtual Memory – A technique that uses main memory as a "cache" for secondary storage
- Virtual memory implements the translation of a program's address space to physical addresses . This translation process enforces protection of a program's space

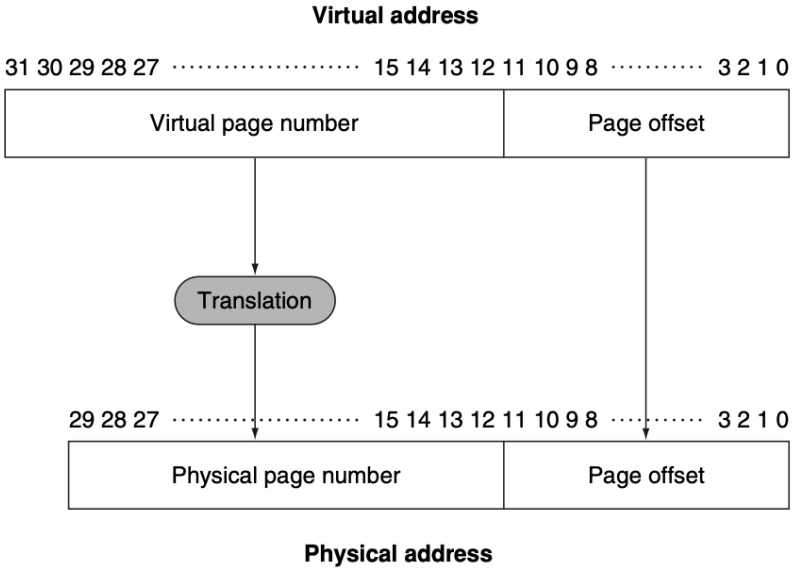
- Physical address – an address in main memory
- This translation process enforces protection of a program's address space from other virtual machines
  - Protection – A set of mechanisms for ensuring that multiple processes sharing the processor, memory, or I/O devices cannot interfere, intentionally or unintentionally, with one another by reading or writing each other's data. These mechanisms also isolate the operating system from a user process
- Second motivation for virtual memory is to allow a single user program to exceed the size of primary memory.
- Multiprogramming goals – transparency, protection, efficiency, sharing

- A virtual memory block is called a page and a virtual memory miss is called a page fault
- Processor produces a virtual address, which is translated by a combination of hardware and software to a physical address, which in turn can be used to access main memory
  - Virtual address – an address that corresponds to a location in virtual space and is translated to a physical address when memory is accessed.



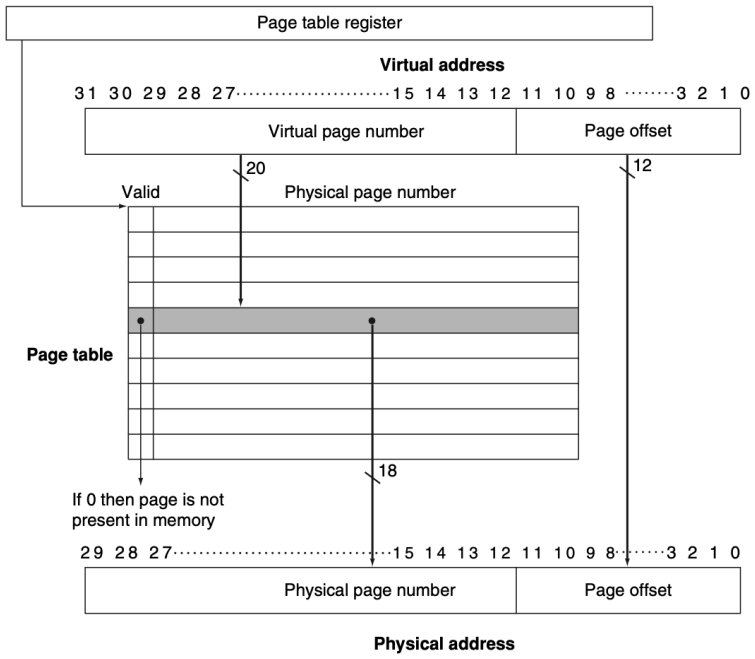
  - Virtual addressed memory with pages mapped to main memory – address mapping
  - Address mapping/translation- the process by which a virtual address is mapped to an address used to access memory
  - Memory blocks of memory (pages) are mapped from one set of addressed (virtual addresses) to another set (physical addresses)

        - When page not mapped to a physical address (not in main memory), the page resides in the disk
- Virtual memory simplifies loading the program for execution by providing relocation



-
    - Mapping from a virtual to a physical address. The page size is 2^{12} = 4 KiB. The number of physical pages allowed in memory is 2^{18}, since the physical page number has 18 bits in it
    - Page offset not translated
- Having a large number of virtual pages than physical pages is the basis for illusion of an essentially unbounded amount of virtual memory
- Page faults have high cost, main memory latency is 100 000 times quicker than disk.
    - Pages should be large enough to try amortize the high access time. Sizes from 4KiB to 16KiB are typical
    - Primary technique used is to allow fully associative placement of pages in memory
    - Write through will not work for virtual memory, write back systems are used
    - Page faults can be handled in software because the overhead will be small compared to the disk access time
- Placing a page and finding it again
    - In virtual memory systems, we locate pages by using a table that indexes the memory; the structure is called a page table and it resides in memory

- A page table is indexed with the page number from the virtual address to discover the corresponding physical page number from the virtual address to discover the corresponding physical page number.
- Each program has its own page table, which maps the virtual address space of that program to main memory
- Page table - The table containing the virtual to physical address translations in a virtual memory system. The table, which is stored in memory, is typically indexed by the virtual page number; each entry in the table contains the physical page number for that virtual page if the page is currently in memory
- To indicate the location of the page table in memory, the hardware includes a register that points to the start of the page table, the page table register

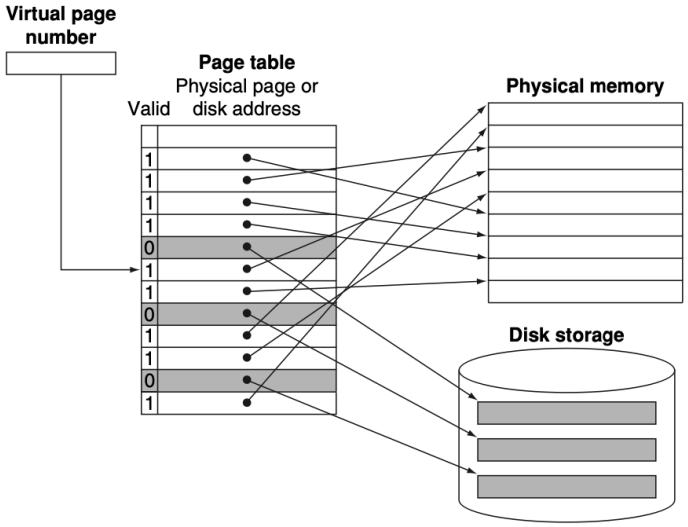


-
  - A valid bit is used in each page to indicate in the page is in page table. Page miss occurs if not in
  - 22 bits => 4 million pages

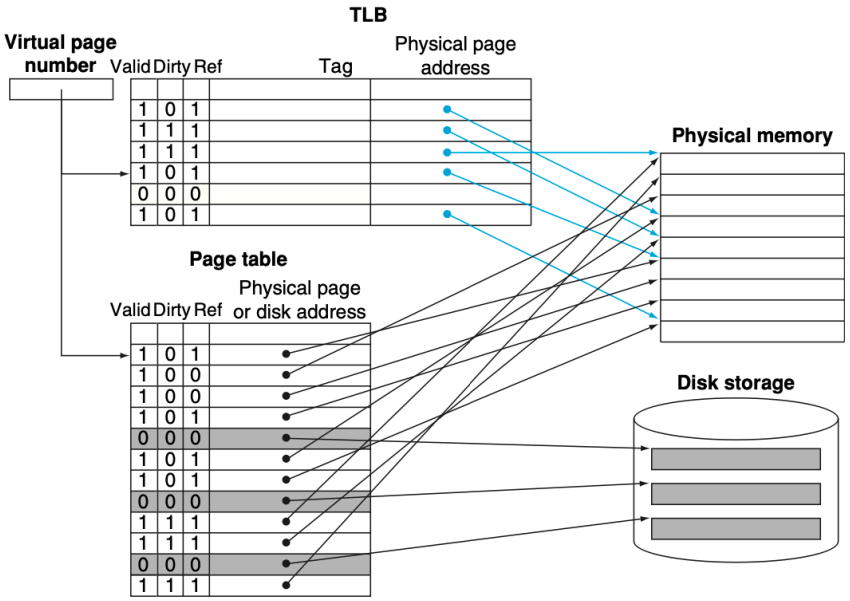
FIGURE 5.27 The page table is indexed with the virtual page number to obtain the corresponding portion of the physical address. We assume a 32-bit address. The page table pointer gives the starting address of the page table. In this figure, the page size is 2^{12} bytes, or 4 KiB. The virtual address space is 2^{32} bytes, or 4 GiB, and the physical address space is 2^{30} bytes, which allows main memory of up to 1 GiB. The number of entries in the page table is 2^{20}, or 1 million entries. The valid bit for each entry indicates whether the mapping is legal. If it is off, then the page is not present in memory. Although the page table entry shown here need only be 19 bits wide, it would typically be rounded up to 32 bits for ease of indexing. The extra bits would be used to store additional information that needs to be kept on a per-page basis, such as protection.

  -

- Page Faults
  - If the valid bit for a virtual page is off, a page fault occurs. The operating system must be given control. This is done with exception mechanism
  - Because it is not known ahead of time when a page in memory will be replaced the operating system usually creates the space on a flash memory or disk for all the pages of a process when it creates the process. The space is called the swap space
    - Swap space – the space on the disk reserved for the full virtual memory space of a process
  - The operating system creates a data structure that tracks which processes and which virtual addresses use each physical page
  - This data structure may be part of the page table or may be an auxiliary data structure indexed in the same way as the page table



    - Pages in main memory and the pages on disk are the same size
  - OS follows the least recently used (LRU) replacement
  - Replaced pages are written to swap space on the disk.
  - Implementing a completely accurate LRU scheme is too expensive, since it requires updating a data structure on every memory reference. Most operating systems apply an approximate LRU
  - To help the OS estimate the LRU pages some computers provide a reference bit or use bit

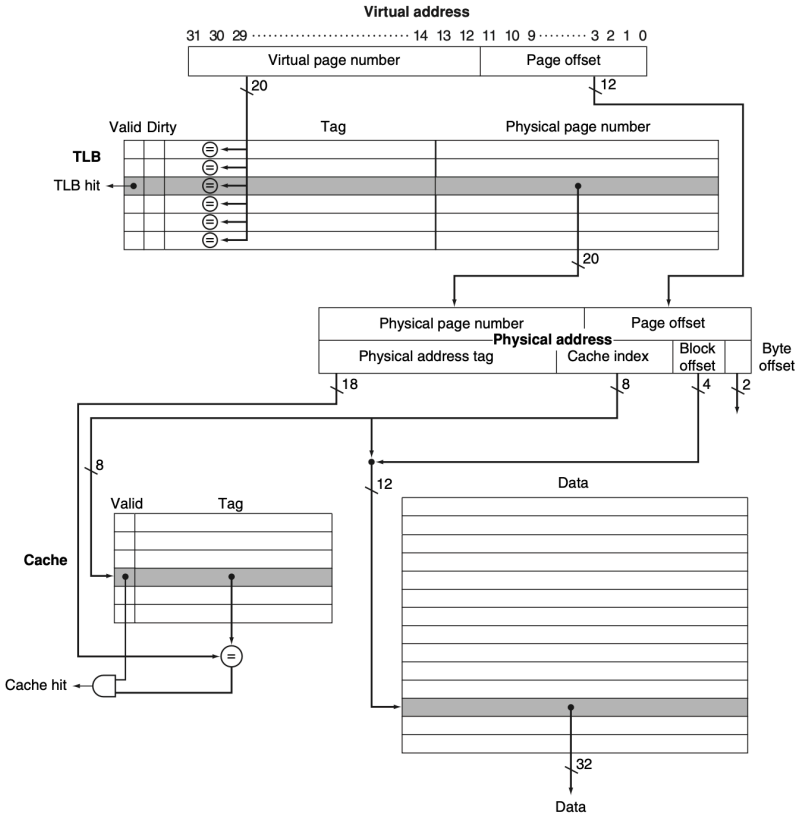
        - Reference bit/ use bit: A field that is set whenever a page is accessed and that is used to implement LRU or other replacement schemes
- Handling Writes
    - The TLB (Translation lookaside buffer)
        - A cache that keeps track of recently used address mappings to try avoid an access to the page table
        - Keeps track of recently used translations
        - 

        - The TLB contains a subset of virtual to physical page mappings that are in the page table.
        - If there is no matching entry in the TLB for a page, the page table must be examined. The page table either supplies a physical page number for the page (which can be used to build a TLB entry) or indicates the page resides on a disk, page fault.
        - The TLB will need to include other status bits, such as dirty and the reference bits. If a hit is obtained the reference bit is turned on. If the processor is performing a write the dirty bit is also turned on.
        - If there is a TLB miss it must be determined if it is a TLB miss or a page fault

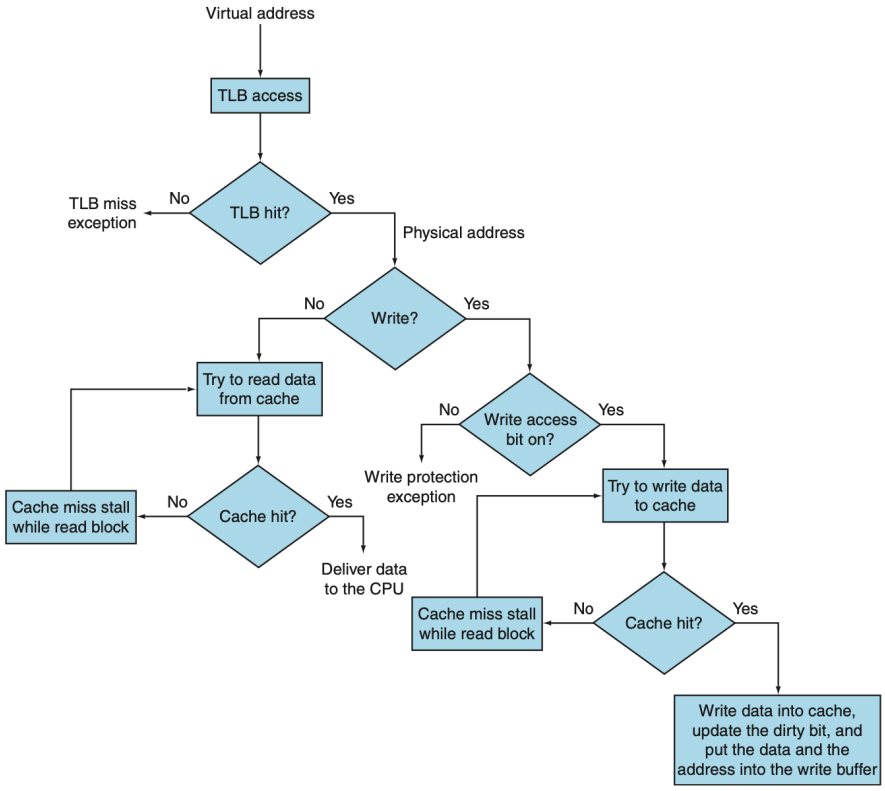
Some typical values for a TLB might be

- TLB size: 16–512 entries
- Block size: 1–2 page table entries (typically 4–8 bytes each)
- Hit time: 0.5–1 clock cycle
- Miss penalty: 10–100 clock cycles
- Miss rate: 0.01%–1%
- TLB status bits are also permission bits

- Integrating Virtual Memory, TLBs and Caches



- The TLB and cache implement the process of going from a virtual address to a data item.



- Protection
  - The protection is done via permissions bits
  - Separate virtual address space for different programs
- Handling TLB misses and Page Faults
  - TLB Miss means: Page is in main memory and only need to create TLB entry
  - Page is not in main memory and we need a transfer control to operating system for page fault

Exceptions and Interrupts

- Exception – An unscheduled event that disrupts program execution; used to detect overflow (TLB miss, syscall)
- Interrupt – An exception that comes from outside the processor, e.g. I/O interrupt (network packet arrived)

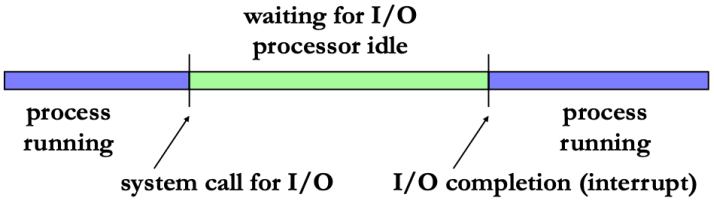
Type of event	From where?	MIPS terminology
I/O device request	External	Interrupt
Invoke the operating system from user program	Internal	Exception
Arithmetic overflow	Internal	Exception
Using an undefined instruction	Internal	Exception
Hardware malfunctions	Either	Exception or interrupt

- Exception mechanism
  - Step 1: Save the address of current instruction
    - Into a special register, the exception program counter (EPC)
    - Note: usually need to resume the interrupted instruction (not PC+4)
  - Step2: Transfer control to the OS to a known address (i.e. exception handler PC)
  - Step 3: Handle exception
    - Deal with the cause
    - All registers must be preserved
  - Step 4: return to user program execution
    - Handler restores user program's registers and jumps back using EPC
    - Relies on special instruction eret
- Finding Exception Handler
  - Approach 1
    - Jump to a predefined address (e.g. 0x800000180 in MARS)
    - Use the cause register to then branch to the right handler (e.g. print int, read string, exit program)
    - Works well for syscall – cause register explicitly set
    - Examples
      - Putc()
      - Send()
      - Sbrk()
  - Apoproach 2
    - Directly jump to a specific handler depending on the exception (vectored interrupt)
    - The interrupt comes from outside (e.g. via CPU pins)
    - Vectored interrupt- An interrupt for which the address to control is transferred is determined by the cause exception
Exception type	Exception vector address (in hex)
Undefined instruction	$8000\ 0000_{hex}$
Arithmetic overflow	$8000\ 0180_{hex}$
    - The OS knows the reason for the exception by the address at which is initiated

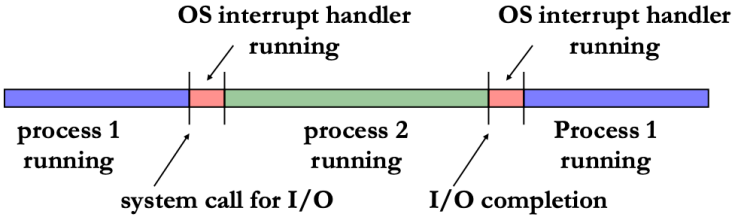
        - Two registers are added onto the MIPS implementation
        - EPC – A 32 bit register used to hold the address of our affected instruction
        - Cause – A register used to record the cause of the exception
- Handling the exception
        - Determine action required
            - By inspecting the Cause register or by virtue of being at the right handler (e.g., undefined opcode)
        - If restartable
            - Take corrective action, then use EPC to return to program
        - Otherwise:
            - Terminate program and report error using EPC
        - For a critical time when the interrupt is being handled, other interrupts should not happen
            - Otherwise the EPC, Cause will be overwritten
            - This is forced by masking interrupts -> by setting the exception level (EXL) bit in the status register
- Protecting System
    - The OS must guarantee safe and orderly access to critical system resources
        - Hardware (processor, networking, I/O)
        - Program memory (including page tables)
    - The OS is the ultimate arbiter of what's allowed
        - TLB miss -> OK (but must access page table to service)
        - Arithmetic overflow -> may be OK (depends on what we're doing)
        - Illegal opcode -> not OK (kill the program)
    - Exceptions are used to hand control over to the OS
        - Need a separate mechanism to limit capabilities of user programs
- Kernel vs User mode protection
    - Exceptions (including syscalls) are handled by the OS
        - CPU has 2 modes of operation: user and kernel (OS)
        - Kernel mode is trusted; user mode is untrusted

        - Current mode identified by a bit in the process status register
    - Privileged instruction only executed in kernel mode
        - Accessing I/O devices, handling page table and TLB updates, halt or reset the processor or change its voltage
    - Kernel mode can only be entered through an exception
        - User programs cannot jump to OS instruction space
    - eret instruction sets mode back to previous mode
- Advantages of Dual Mode architecture
    - Guarantees that control is transferred to OS when user programs attempt to perform potentially dangerous tasks
    - Allow OS to ensure that programs do not interfere with each other
        - Each program is able to get its share of physical memory
    - Allow OS to ensure that programs do not have access to resources which they do not have permissions
        - E.g. files another program's memory
    - Ensure that user programs do not have indefinite control of the processor
- Time sharing of the CPU
    - Issues:
        - Multiple programs, one program
        - I/O takes too long -> processor idle
        - User programs can crash or monopolize the CPU (either unintentionally or maliciously)
    - Solution
        - Multiplex or time-share the CPU and other resources among several user processes
        - Switch from one process to another when it performs I/O or when its time allocation expires

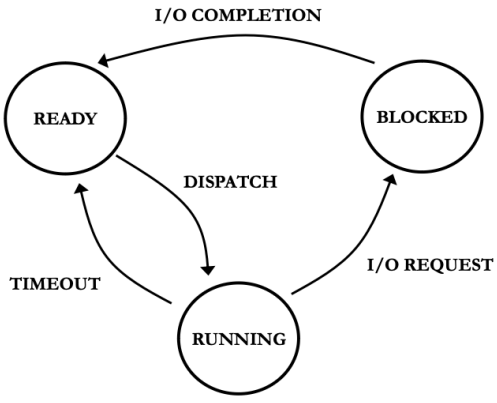
Single-task system:



Multi-tasking system:



- Managing process (a program in execution)
  - Processes are managed by the OS kernel
    - Kernel: the core of the operating system that controls all software and hardware resources
      - First to be loaded when computer boots
      - Manges interrupts, processes, memory, I/O
  - New processes can be explicitly created by the user or implicitly by another process (through forking)
    - Original process -> parent
    - New process -> child
- Process states



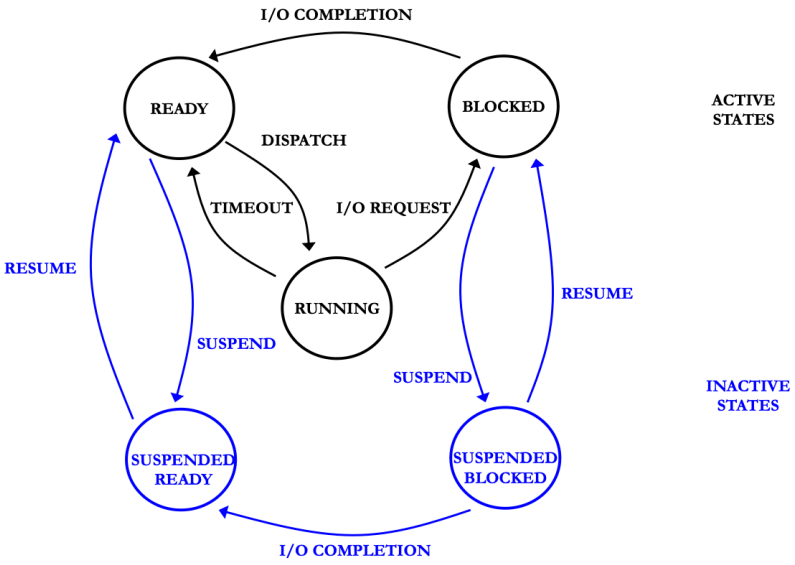
States:

RUNNING: process is currently running in the CPU
READY: process is not running, but could run if brought into CPU
BLOCKED: process is not able to run because it is waiting for I/O to finish

Transitions:

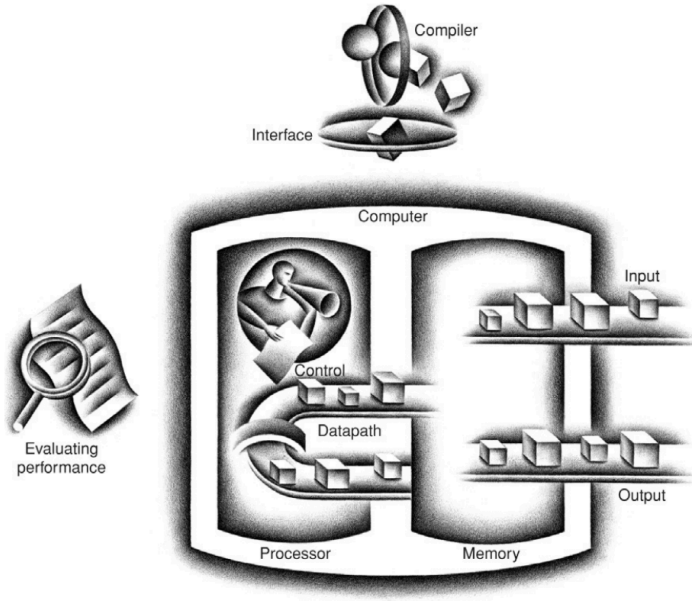
I/O REQUEST: process initiates I/O
I/O COMPLETION: I/O finishes
DISPATCH: OS moves process into CPU and it starts executing
TIMEOUT: process's timeslice is over

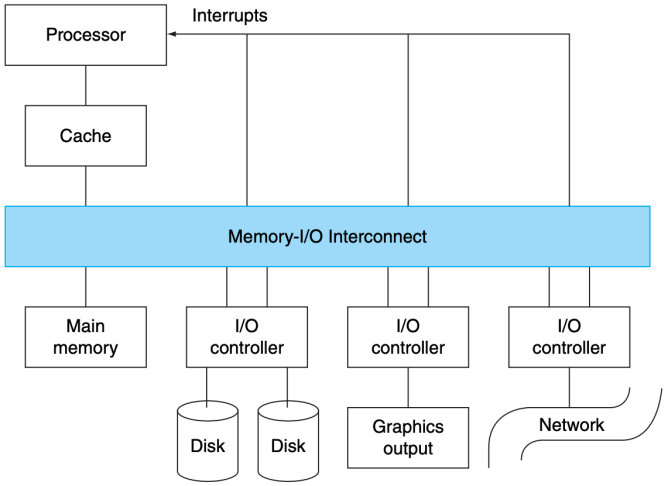
  - Step 1: process calls (or traps into) the OS or interrupt occurs (e.g. because of a timer)
  - Step 2: OS's dispatcher performs a context switch:

        - Process's context is saved (registers, PC, etc.) in a process control block (PCB)
        - Dispatcher chooses a new process to run
        - Processes' states are updated
- PCB: OS data structure containing each process's info
    - Process id (PID)
    - Process state (blocked, running, etc)
    - Process priority
    - Process perms
    - Etc
- Suspending and Resuming Processes
    - Problem:
        - Might not have enough physical memory for all processes are still kept in OS memory
        - Inactive processes are resumed by swapping in the data from disk back to memory
        - 

Input/Output

The Five Classic Components of a Computer

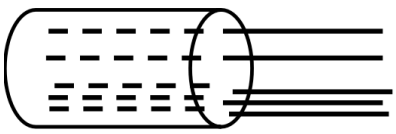




- Structure of simple system with I/O
- The connections between the I/O devices and processor and memory are called buses, although the term means shared parallel wires are closer to dedicated serial lines

- Three characteristics useful in organizing this wide variety:
  - Behaviour: Input (read once), output (write only, cannot be read), or storage (can be reread and usually rewritten)
  - Partner: Either human or machine is at the other end of the I/O device either feeding data on input or reading data on output
  - Data rate: the peak rate at which data can be transferred between the I/O device and the main memory or processor. It is useful to know the maximum demand the device may generate when designing an I/O system

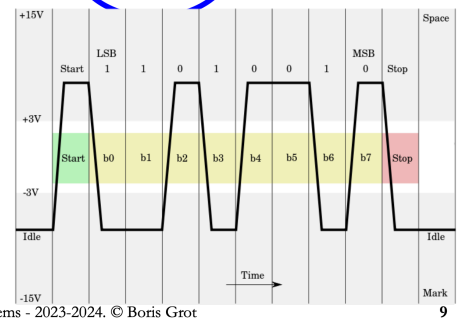
Device	Behavior	Partner	Data rate (Mbit/sec)
Keyboard	Input	Human	0.0001
Mouse	Input	Human	0.0038
Voice input	Input	Human	0.2640
Sound input	Input	Machine	3.0000
Scanner	Input	Human	3.2000
Voice output	Output	Human	0.2640
Sound output	Output	Human	8.0000
Laser printer	Output	Human	3.2000
Graphics display	Output	Human	800.0000–8000.0000
Cable modem	Input or output	Machine	0.1280–6.0000
Network/LAN	Input or output	Machine	100.0000–10000.0000
Network/wireless LAN	Input or output	Machine	11.0000–54.0000
Optical disk	Storage	Machine	80.0000–220.0000
Magnetic tape	Storage	Machine	5.0000–120.0000
Flash memory	Storage	Machine	32.0000–200.0000
Magnetic disk	Storage	Machine	800.0000–3000.0000

        - Diversity of I/O devices
- I/O requests – Reads or Writes to I/O devices
- Example- RS232 Serial Interface
    - I/O controller: UART
        - Universal Asynchronous Receiver Transmitter
    - Used for modems and other serial devices
    - Physical implementation:
        - Two data wires (one for each direction) + ground reference + status signals
        - 

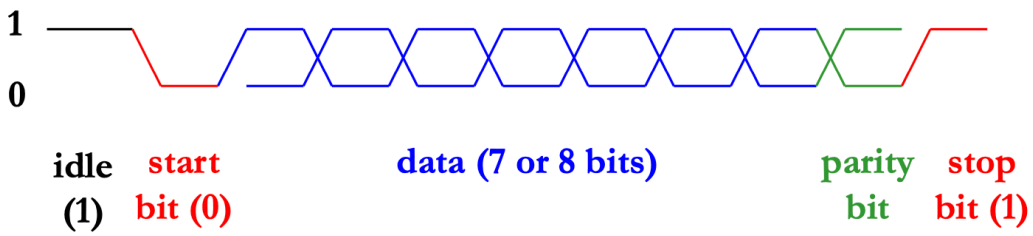
RS232 modem: bits and wires

Oscilloscope trace of voltage levels for an ASCII "K" character (0x4B) with 1 start bit, 8 data bits, 1 stop bit. (Note: not RS232!)

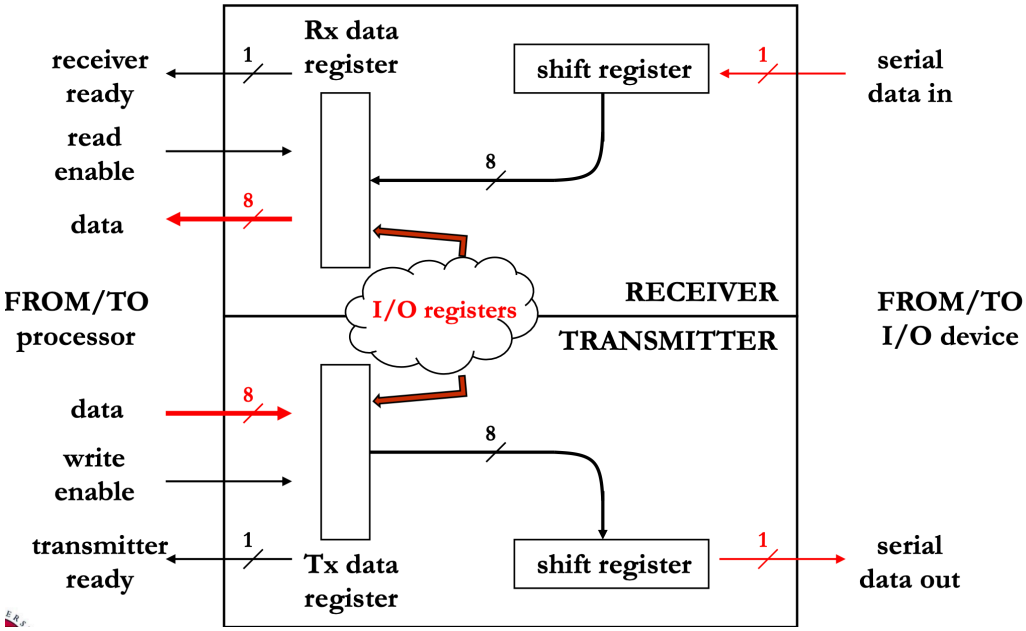
Source: wikipedia



- Encoding
  - 1 ASCII character = 11 or 10 bits (including signalling)
  - Idle state represented by a constant 1
- Parity: A 1 bit for detection of transmission errors
  - Odd parity ->total number of 1's (including parity bit) is odd
  - Even parity ->total number of 1's is even



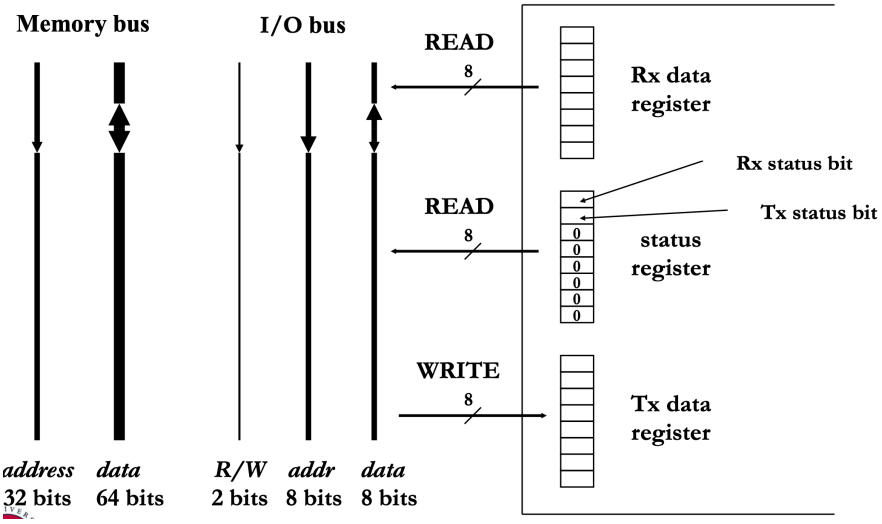
- UART controller



- Connecting Processors, Memory and I/O devices
  - I/O transaction – A sequence of operations over the interconnect that includes a request and may include a response, either of which may carry data. A transaction initiated by a single request and may take many individual bus operations.
  - I/O transactions includes two parts: sending the address and receiving or sending the data
  - Bus transactions are defined by what they do to memory
  - *Read* transfers data from memory, *write* writes data to memory
  - Standard for expanding the machine and connecting new peripherals

Characteristic	Firewire (1394)	USB 2.0	PCI Express	Serial ATA	Serial Attached SCSI
Intended use	External	External	Internal	Internal	External
Devices per channel	63	127	1	1	4
Basic data width (signals)	4	2	2 per lane	4	4
Theoretical peak bandwidth	50 MB/sec (Firewire 400) or 100 MB/sec (Firewire 800)	0.2 MB/sec (low speed), 1.5 MB/sec (full speed), or 60 MB/sec (high speed)	250 MB/sec per lane (1x); PCIe cards come as 1x, 2x, 4x, 8x, 16x, or 32x	300 MB/sec	300 MB/sec
Hot pluggable	Yes	Yes	Depends on form factor	Yes	Yes
Maximum bus length (copper wire)	4.5 meters	5 meters	0.5 meters	1 meter	8 meters
Standard name	IEEE 1394, 1394b	USB Implementors Forum	PCI-SIG	SATA-IO	T10 committee

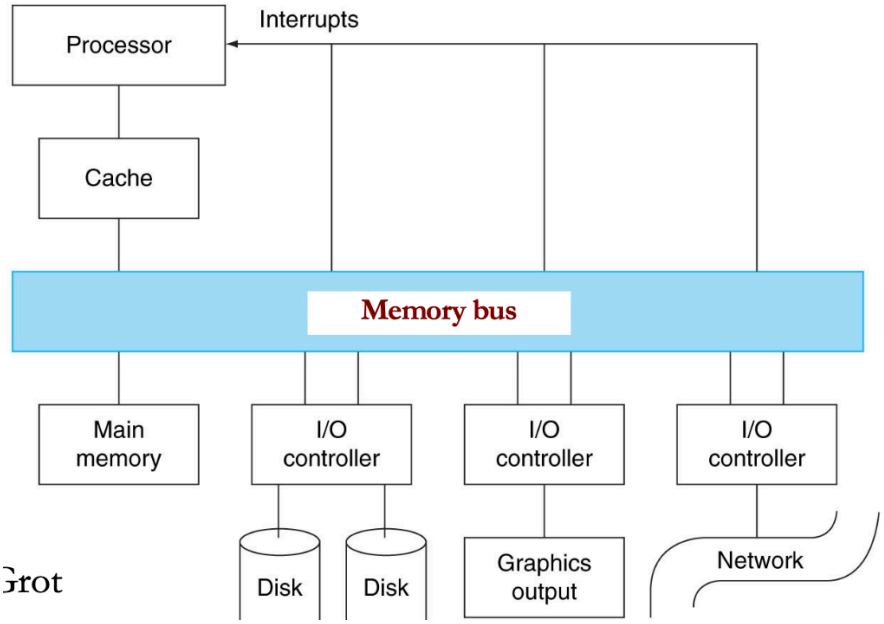
- Synchronous buses – A bus that includes a clock in the control in the control lines and a fixed protocol for communicating that is relative to the clock – used in traditional buses – can be easily implemented in a small fsm
- Asynchronous interconnect – Uses a handshaking protocol for coordinating usage rather than a clock; can accommodate a wide variety of devices of differing speeds
- Handshaking protocol – A series of steps used to coordinate asynchronous bus transfers in which the sender and the receiver proceed to the next step only when both parties agree that the current step has been completed
- Option 1: connect the I/O Tx and Rx registers directly into dedicated CPU I/O registers -> not flexible, not expandable
- Option 2: keep I/O registers in a separate I/O controller and connect CPU to I/O controller through special I/O bus
    - Requires a separate interface for I/O, potentially expensive
    - I/O bus:
        - Data lines (8 lines)
        - Control lines (READ and WRITE signals)
        - Address lines -> each I/O controller is assigned a range of addresses for its registers
        - Data is accessed through special I/O loads and stores



- o Option 3: Keep I/O registers in I/O controller and connect the CPU to I/O controller through the memory bus
  - Memory-mapped I/O:
    - An I/O scheme in which portions of address space are assigned to I/O devices and reads and writes to those addresses are interpreted as commands to the I/O device
    - When the processor places the address and data on the memory bus the memory system ignores the operation because the address indicates a portion of memory used for I/O. The device controller sees the operation, records the data and transmits it to the device as a command.
    - Memory-mapped I/O can also be used to transmit data by writing or reading to select addresses. The device uses the address to determine the type of command and the data may be provided by a write or obtained by a read
    - Address encodes device identity and type of transmission
    - Requests may take several I/O operations e.g. printer contains 2 registers, contains a done bit, set by the printer when it has printed a character and an error bit, indicating that the printer is jammed or out of paper. Each byte of data to be printed is put into data

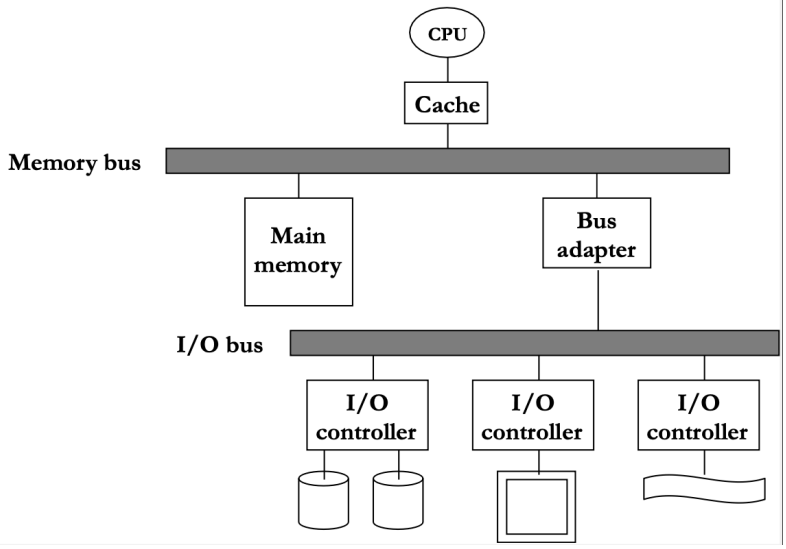
register. The processor must then wait until the printer sets the done bit before it can place another character in the buffer

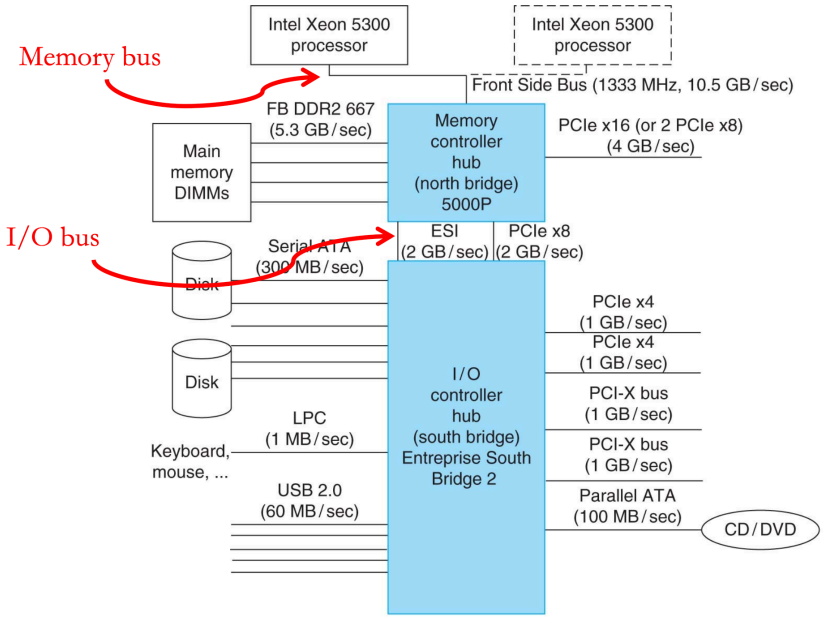
- I/O controller registers (data and control) are mapped to a dedicated portion of memory, accessed via the memory bus
- Pros: accessed with regular load and store instructions
- Cons: Takes away bandwidth away from CPU <-> Memory



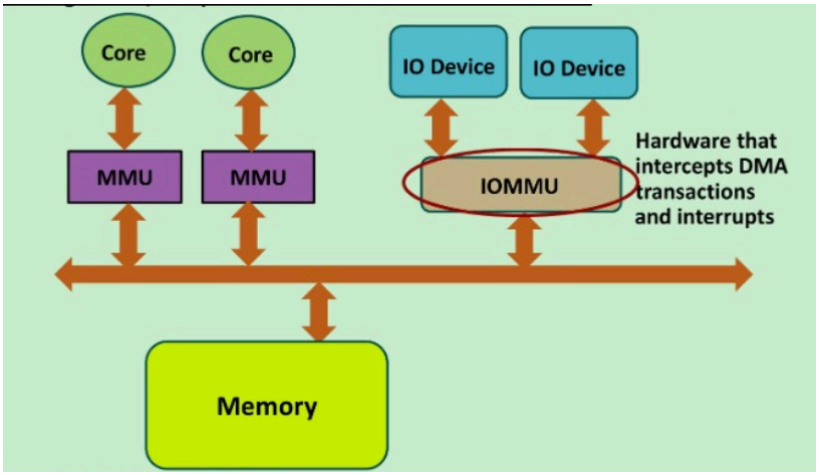
- Grot

- Option 4: connect I/O controllers to I/O bus and the I/O bus to the memory bus through a bus adapter
  - Off-load the I/O from the memory bus: multiple I/O devices appear as a single device to the memory bus
  - Pros:
    - Better performance: slow & narrow I/O bus doesn't clog up the fast memory bus
    - Higher flexibility: add/remove devices without impacting the high performance processor memory interface
    - Modularity and extensibility: memory bus and I/O bus technology can evolve separately





- Processor connects peripherals via two main chips
- North bridge is basically a DMA controller, connecting the processor, graphics card and the south bridge chip
- The south bridge connects the northbridge to a cornucopia of I/O buses

- 

    - Approach used in today's systems
- A bus or network protocol defines how a word or block of data should be communicated on a set of wires. This still leaves other tasks that must be performed to actually cause data to be transferred from a device into memory address space of some user program.
- Operating system responsibilities arise from three characteristics of I/O systems:
    - Multiple programs using the processor share the I/O system
    - I/O systems often use interrupts to communicate information about I/O operations. Because interrupts cause a transfer to kernel or supervisor mode, they must be handled by the OS
    - Low level control of an I/O device is complex, because it requires managing a set of concurrent events and because the requirements for correct device control are often very detailed
- Three characteristics of I/O systems above lead to several different function the OS must provide:
    - OS guarantees that a user's program accesses only the portions of an I/O device to which the user has rights.
    - The OS provides abstractions for accessing devices by supplying routines that handle low-level device operations
    - The OS handles the interrupts generated by I/O devices, just as it handles the exceptions generated by a program
    - The OS tries to provide equitable access to the shared I/O resources as well as schedule accesses to enhance system throughput

- To perform these functions on behalf of user programs, the operating system must be able to communicate with the I/O devices and to prevent the user program from communicating with the I/O devices. Three types of communication are required:
    - The OS must be able to give commands to the I/O devices. These commands include not only read and write, but other operations to be done on the device such as disk seek
    - The device must be able to notify the OS when the I/O device has completed an operation or has encountered an error
    - Data must be transferred between memory and an I/O device
- Communicating with the Processor
    - Polling- the process for periodically checking the status of an I/O device to determine the need to service the device
        - Device simply puts the information in a status register, and the processor must come get the information. Processor is totally in control and does all the work
        - Real-time embedded applications poll the I/O, since the I/O rates are predetermined and it makes I/O overhead more predictable, which is helpful for real time
        - Disadvantage of polling is it can waste a lot of processor time, because processors are so much faster than I/O devices e.g. a mouse not moving
    - Interrupt-driven I/O – An I/O scheme that employs interrupts to indicate the process that an I/O device needs attention
        - An I/O interrupt is asynchronous with respect to the instruction execution, not associated with any instruction and does not prevent the instruction completion
        - In addition to the fact an I/O interrupt has occurred, the identity of the device generating the interrupt must be know. The interrupts represents devices that may have different priorities and whose interrupt requests have different urgencies associated with them
        - To communicate information to the processor vectored interrupts or an exception cause register can be used
        - Interrupt Priority Levels
            - Steps to handling an interrupt:

    - Logically AND the pending interrupt field and the interrupt mask field to see which enabled interrupts could be the culprit. Copies are made of these two registers using the mcf0 instruction
    - Select the higher priority of these interrupts. The software convention is that the leftmost is the highest priority
    - Save the interrupt mask field of the Status register
    - Change the interrupt mask field to disable all interrupts of equal or lower priority
    - Save the processor state needed to handle the interrupt
    - To allow higher-priority interrupts, set the interrupt enable bit of the Cause register to 1
    - Call the appropriate interrupt routine
    - Before restoring state, set the interrupt enable bit of the Cause register to 0. This allows you to restore the interrupt mask field

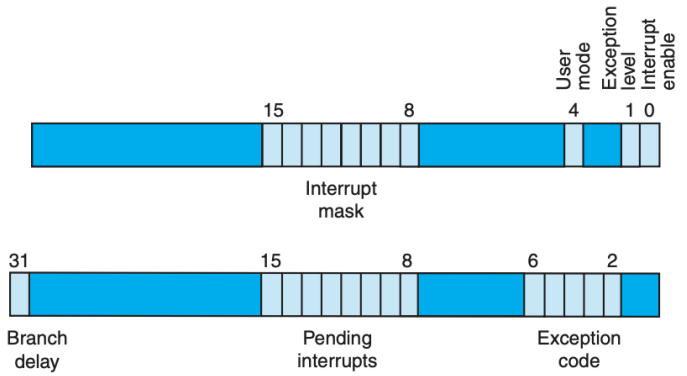


FIGURE 6.11 The Cause and Status registers. This version of the Cause register corresponds to the MIPS-32 architecture. The earlier MIPS I architecture had three nested sets of kernel/user and interrupt enable bits to support nested interrupts. Section B.7 in Appendix B has more details about these registers.

-
    - The cause and status registers
- At the lowest IPL, all interrupts are permitted. Conversely, at the highest IPL, all interrupts are blocked. Raising and lowering IPL involved changes to the interrupt mask field of the status register

- USB example

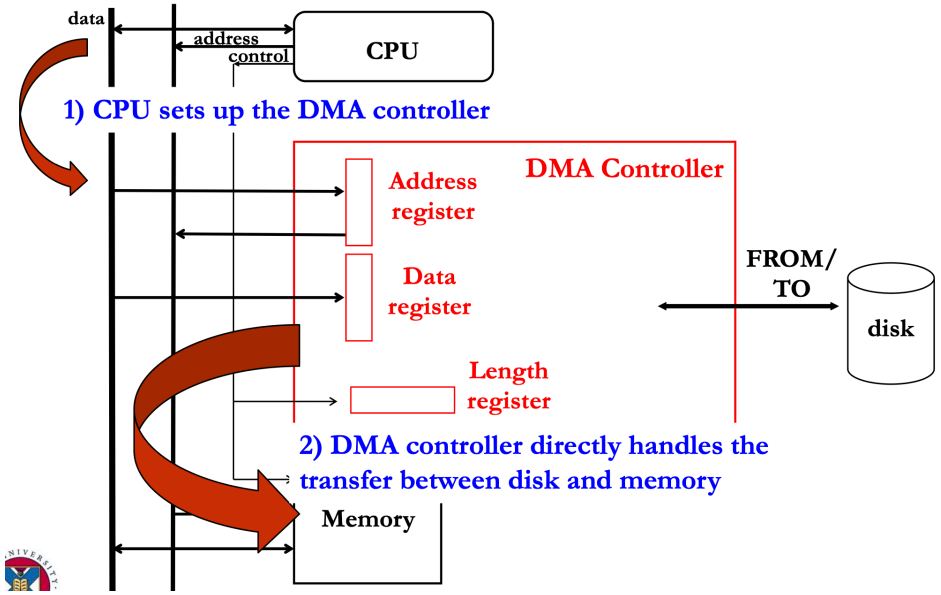
    - USB doesn't generate interrupts
        - Keeps costs low
    - Computer can't afford to keep polling the multitude of USB ports
        - Would lose too much performance for nothing
        - Not polling often enough not an option: will lose
    - USB controller does the polling (via fsm) and generates an interrupt to inform the CPU as needed
        - This functionality is in the SouthBridge
- Hard disks
    - Disk storage is non-volatile
        - Non-volatile-storage device where data retains power its value even when power is removed
    - A magnetic disk consists of a collection of platters (1-4), each of which has two recordable disk surfaces. The stack of platters is rotated at 5400 to 15 000 RPM and has a diameter from 1 inch to just over 3.5 inches. Each disk is divided into concentric circles called tracks. Each track is divided into sectors, each track may have about 100 to 500 sectors. Each sector is typically 512 bytes in size but may increase to 4096 bytes
    - To read and write information the read/write heads must be moved so that they are over the correct location. The disk heads are connected for each surface so that every head is over the same track of every surface. Cylinder is used to refer to all the tracks
    - To access data, the operating system must direct the disk through a three stage process. First stage is seek
        - Seek – the process of positioning a read/write head over the proper track on a disk
    - Once the head has reached the correct track, we must wait for the desired sector to rotate under the read/write head. This time is called the rotational latency or rotational delay.
        - Rotational latency. Also called rotational delay. – the time required for a desired sector of a disk to rotate under read/write head; usually assumed to be half the rotation time

    - Transfer time is the time to transfer a block of bits. It is a function of sector size, the rotation speed and the recording density of a track.
    - A disk controller usually handles the detailed control of the disk and the transfer between disk and the memory
    - Advanced Technology Attachment and Small Computer Systems Interface – A command set used as a standard for I/O devices
    - Access time = seek time + rotational latency
    - Transfer time = time to transfer 1 bit * number of bytes of data
        - Dependent on spinning speed and recording density
- Disk Controllers
    - A disk controller inside disk unit -> responsible for all mechanical operation of disk + interface with CPU
    - I/O registers:
        - Exchange data and status words between CPU and disk controller
        - Command register -> tells disk controller what to do next e.g. Seek n; Read sector m; Write sector m; format track -> High level commands
    - Using disk controller
        - Step 1: user program requests data from file
        - Step 2: OS determines sector to be accessed
        - Step 3: OS issues Seek command and CPU goes to work on other processes (multi-tasking)
        - Step 4: I/O controller interrupts CPU to signal completion of seek
        - Step 5: OS disk handler issues Read Sector command and CPU goes to work on some other process
        - Step 6: I/O controller interrupts CPU to signal data ready
        - Step 7: OS disk handler transfers sector data from disk controller to memory
        - This is a slow running for loop that transfers data word for word via memory buses
        - Step 8: go to step 3 or step 5 and repeat until all data transferred

    - Interrupt-driven I/O relieves the processor from having to wait for every I/O event, although if we used this method for transferring data from or to a hard disk, the overhead could still be intolerable, since it could consume a large fraction of the processor when the disk is transferring
    - CPU (through OS) also has to issue individual commands to read every sector from disk
    - Frequent interrupts detrimental to processor performance
        - Frequent switching between OS and application
    - CPU moving the data from the disk controller via the slow I/O bus results in highly inefficient CPU utilization
- Direct memory access (DMA)- A mechanism that provides a device controller with the ability to transfer data directly to or from memory without involving the processor
    - Setup by the processor for each transfer using memory-mapped registers inside the DMA controller
- The interrupt mechanism is still used by the device to communicate with the processor, but only on completion of the I/O transfer or when an error occurs
- DMA registers
    - Address registers -> position in memory of next data to be read/written
    - Data register-> temporary storage for data to be transferred
    - Length register-> number of bytes remaining to be transferred
- The DMA controller becomes the master and directs the reads or writes between itself and memory
    - Master – a unit on the I/O interconnect that can initiate transfer requests
- DMA transfer
    - Step 1: the processor sets up the DMA by supplying the identity of the device, the operation to perform on the device, the memory address that is the source or destination of the data to be transferred and the number of bytes to transfer
    - Step 2: the DMA starts the operation on the device and arbitrates for the interconnect. When the data is available (from the device or memory), it transfers the data. The DMA device supplies the

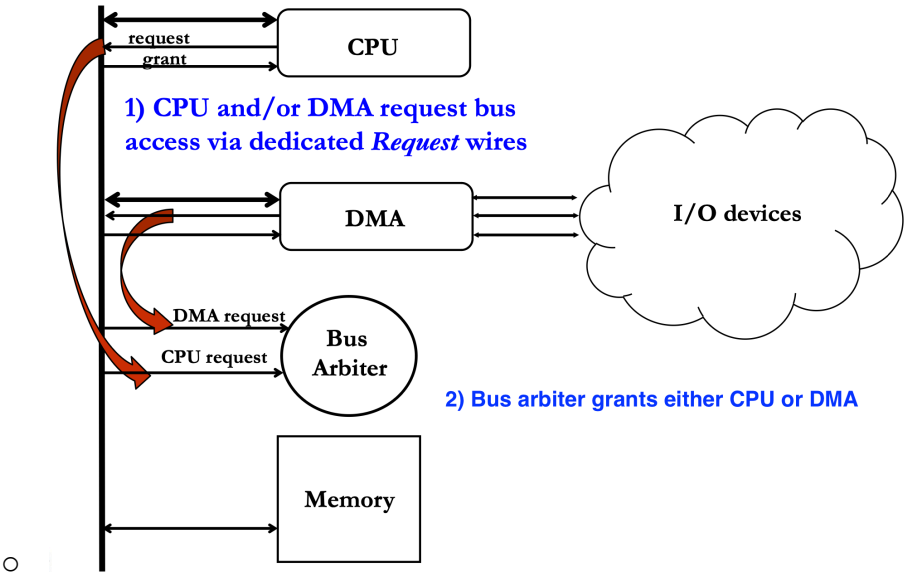
memory address for the read or the write. If the request requires more than one transfer the DMA unit generates the next memory address and initiates the next transfer. Using this mechanism, a DMA can complete an entire transfer, which may be thousands of bytes in length, without bothering the process. Many DMA controllers contain some memory to allow them to deal flexibly either with delays in transfer or with those incurred while waiting to become the master

- Step 3: Once the DMA transfer is complete, the controller interrupts the processor which can then determine by interrogating the DMA device or examining memory whether the entire operation completed successfully



- There may be multiple DMA devices in a computer system
- DMA operation
  - Step 1: user program requests data from file
  - Step 2: OS determines sector to be accessed
  - Step 3: OS issues Seek command and CPU goes to work on other processes (multi-tasking)
  - Step 4: I/O controller interrupts CPU to signal completion of seek
  - Step 5: OS disk handler issues Read Sector command and CPU goes to work on some other process
  - Step 6: I/O controller informs DMA controller that the data is ready (no need to interrupt CPU)

        - Step 7: DMA controller transfers data into memory; length register is decremented until all the data is moved (advanced DMA controllers can access multiple tracks with a single operation)
        - Step 8: DMA controller interrupts CPU to inform completion of DMA operation
    - When DMA is incorporated into I/O system the relationship between the memory system and processor changes i.e. stale data problem or coherence problem – multiple copies of a data item, one in cache and in memory; versions of the memory in cache vs that in memory
- Paging in MIPS
    1. TLB Miss
    2. Trap to Kernel
    3. Access Page Table
    4. Find page invalid
    5. Convert virtual address to file + offset
    6. Allocate a page (update page table) – Evict page if needed
    7. Initiate DMA transfer
    8. Disk interrupt when DMA transfer is complete
    9. Mark page as valid
    10. Load TLB entry
    11. Resume process at faulting instruction
- Bus arbitration
    - DMA and CPU both have access to memory bus -> access must be arbitrated to avoid conflicts
        - I/O devices are slaves and cannot initiate bus transactions
    - Solution: Additional Logic (bus arbiter)
        - Authorizes CPU or DMA to access memory at any given time
    - Two new wires on the memory bus
        - Bus request- asserted by the CPU or the DMA when it requires the bus
        - Bus grant – asserted to indicate who (CPU or DMA can use the bus)
        - In case of conflicting requests in the same cycle, CPU usually has priority


request
grant
CPU
1) CPU and/or DMA request bus access via dedicated *Request* wires
DMA
I/O devices
DMA request
CPU request
Bus Arbiter
2) Bus arbiter grants either CPU or DMA
Memory